DE

Data Engineer Interview Preparation

Senior / Lead Data Engineer — Complete Course & Question Bank

Atresh Somayajula

Senior Data Engineer · Azure · Databricks · Delta Lake · Fabric

11 chapters · SQL · Python/PySpark · Spark/Delta · Modeling · Pipelines · Azure/Fabric · Gap Areas · System Design · Behavioral · Mock Loops

Compiled June 25, 2026

Built from analysis of 244 Data Engineer applications. Prepared for interview study use.

Table of Contents

Data Engineer Interview Prep Course — Atresh Somayajula

1. What the market is actually asking you (evidence, not guesswork)
2. The roles you're interviewing for (and how they interview)
3. The 5 interview rounds you should prepare for
4. Course map
5. A 4-week study plan (≈10 hrs/week)
6. How to use each module
7. Your 60-second positioning statement (memorize)

Module 01 — SQL Mastery

- Topics to prepare
- The 5 patterns interviewers reward
- Frequently asked questions (rapid Q&A)
- Mock SQL problems (with full answers)
- Performance talking points (say these, you'll stand out)

Module 02 — Python & PySpark

- Part A — Python for Data Engineering
- Part B — PySpark
- Pandas ↔ PySpark mental map (say this if asked "you only know pandas?")

Module 03 — Spark Internals, Databricks & Delta Lake

- Part A — Spark architecture & execution
- Part B — Delta Lake internals
- Part C — Databricks platform
- Mock deep-dive (full answers)

Module 04 — Data Modeling & Warehousing

- Topics to prepare
- The non-negotiables (be able to draw these on a whiteboard)
- Frequently asked questions (rapid)
- Mock modeling problems (with answers)

Module 05 — Pipelines, Orchestration & Streaming

- Topics to prepare
- ETL vs ELT (have a crisp answer)
- Idempotency — the senior signal
- Exactly-once vs at-least-once
- Orchestration concepts (Airflow vocabulary you must speak)
- CDC (Change Data Capture)
- Streaming essentials
- Mock pipeline problems (with answers)

Module 06 — Azure, Synapse, Fabric & Event Hub (Your Differentiator)

- Topics to prepare

Service-by-service cheat sheet
Frequently asked questions (rapid)
Reference architecture you can draw (your actual platform)
Mock platform questions (with answers)

Module 07 — Gap Areas: dbt, Snowflake, Scala & Airflow

7.1 dbt (data build tool) — demand 80×
7.2 Snowflake — many dedicated roles
7.3 Scala (for Spark) — demand 172×
7.4 Airflow — demand 112×
Cheat sheet: mapping YOUR experience → the gap tool

Module 08 — Data Engineering System Design

The framework (use this every single time)
The reusable building blocks (your toolbox)
Worked example 1 — Design a data warehouse / analytics platform for an e-commerce company
Worked example 2 — Real-time analytics / metrics pipeline (e.g., live dashboard of events)
Worked example 3 — Ingest from 100s of source tables (metadata-driven framework)
Worked example 4 — CDC pipeline from an OLTP database to the lakehouse
Worked example 5 — Data quality / observability framework
Worked example 6 — Batch + streaming unification (Lambda vs Kappa decision)
Cross-cutting checklist (run through these out loud — earns senior signal)
Common system-design FAQ

Module 09 — Behavioral & STAR (Built From Your Real Projects)

The STAR framework
Your story bank (memorize these 6 — they cover ~90% of questions)
15 behavioral questions with model answers (mapped to your stories)
Amazon Leadership Principles (you applied to Amazon — prep these specifically)
Sponsorship / logistics scripts (you're on H-1B requiring transfer)
Smart questions to ask THEM (always have 4–5)

Module 10 — Full Mock Interviews (Timed, With Model Answers)

MOCK 1 — Fintech Senior Data/Analytics Engineer (60 min)
MOCK 2 — Big Tech Senior Data Engineer (75 min)
MOCK 3 — Data Platform Vendor (Databricks / MongoDB) (60 min)
MOCK 4 — Consulting / Azure Data Engineer (45 min)
Self-scoring rubric (grade each mock 1–5)
Final-week checklist

Appendix A — Gap Analysis: What's Missing & How to Make This a Solid Interview-Prep Pack

A.1 Technical coverage gaps (ranked by interview risk)
A.2 Structural gaps that weaken it as a document
A.3 How this PDF is built (and how to make it even stronger)
A.4 One-page cheat sheet (quick reference)
A.5 Glossary of key terms
A.6 Suggested next deliverables (in order)

Data Engineer Interview Prep Course — Atresh Somayajula

A complete, **data-driven** interview preparation course built by analyzing **244 Data Engineer applications** in your `applications/output/` folder, the job descriptions you applied to, and your own profile in `config/candidate.json`.

> Goal: walk into Senior / Lead Data Engineer interviews able to (a) **ace the topics you already know cold**, and (b) **not freeze** on the gaps the market keeps asking for (Scala, Airflow, dbt, Snowflake).

---

1. What the market is actually asking you (evidence, not guesswork)

Aggregated keyword frequency across your 244 DE-related application packets. "Matched" = the JD asked for it **and** you have it. "Missing" = the JD asked for it and your resume did **not** clearly show it — these are your **interview risk areas**.

Skill	Demand (×)	You have it?	Priority
SQL	231	✅ Strong	**Must ace**
Python	207	✅ Strong	**Must ace**
Spark / PySpark	149 / 54	✅ Strong	**Must ace**
ETL / ELT	159 / 120	✅ Strong (vocabulary gap only)	Must ace
Data pipelines	151	✅ Strong	Must ace
Azure	119	✅ Strong	Differentiator
Databricks	84	✅ Strong	Differentiator
Streaming	69	✅ Strong (Event Hub, Structured Streaming)	High
Data warehouse / lakehouse / lake	65 / 51 / 47	✅ Strong	High
Delta Lake	41	✅ Strong	High
Kafka	43	⚠️ Adjacent (Event Hub)	Medium
Power BI / DAX / semantic models	44 / 8 / 26	✅ Strong	Differentiator
Dimensional modeling / star schema	14 / 8	✅ Strong	High
Scala	**172**	❌ **Gap**	**Close the gap**
Airflow	**112**	❌ **Gap (you use ADF/Workflows)**	**Close the gap**
dbt	**80**	❌ **Gap**	**Close the gap**
Snowflake	many roles	❌ **Gap**	**Close the gap**

Read this twice: Your *weakest* market signals (Scala 172, Airflow 112, dbt 80) are also the *most frequently requested* tools you don't have. Module 07 is dedicated to making you conversant — not expert — in all four, plus exactly how to answer "Have you used X?" honestly and strongly.

---

2. The roles you're interviewing for (and how they interview)

From your applications, you face **four archetypes**, each with a different interview style:

Archetype	Examples from your applications	Interview emphasis
Big Tech	Amazon, Microsoft, Dropbox, Reddit, Roblox, Samsara	SQL + Python/Spark coding, data modeling, **system design**, behavioral (Amazon = LPs)
Fintech / high-growth	Robinhood, SoFi, Brex, Gusto, Instacart, Twilio	SQL + dbt/Snowflake/Airflow, analytics engineering, data quality, stakeholder stories
Data platform vendors	Databricks, MongoDB	Deep Spark/Delta/lakehouse internals, architecture, customer-facing communication
Consulting / staffing	Accenture, Capgemini, TCS, Cognizant, Deloitte, Neon IT	Breadth across Azure/GCP, ETL fundamentals, client scenarios, screening Q&A

---

3. The 5 interview rounds you should prepare for

1. **Recruiter / phone screen** — résumé walkthrough, "tell me about yourself", logistics (sponsorship, comp, location). → Module 09.
2. **Technical screen — SQL + coding** — live SQL, Python/PySpark, maybe a take-home. → Modules 01, 02.
3. **Data modeling & pipeline deep-dive** — dimensional modeling, medallion, Delta, orchestration. → Modules 03, 04, 05, 06.
4. **System design** — "Design X data platform / pipeline." → Module 08.
5. **Behavioral / values** — leadership, conflict, impact, failure (Amazon LPs, etc.). → Module 09.

---

4. Course map

Module	File	Covers
00	this `README.md`	Overview, study plan, requirements analysis
01	01-sql-mastery.md	SQL theory, patterns, 20+ Q&A, coding problems w/ answers
02	02-python-and-pyspark.md	Python for DE, PySpark API, coding problems
03	03-spark-databricks-delta.md	Spark internals, Databricks, Delta Lake
04	04-data-modeling-warehousing.md	Kimball/Inmon, SCDs, medallion, lakehouse
05	05-pipelines-orchestration-streaming.md	ETL/ELT, Airflow, CDC, Kafka, Structured Streaming

Module	File	Covers
06	06-azure-fabric-platform.md	ADF, Synapse, Fabric, Event Hub (your differentiator)
07	07-gap-areas-dbt-snowflake-scala-airflow.md	Crash courses on your 4 gaps + how to answer honestly
08	08-system-design.md	Framework + 6 worked DE system-design problems
09	09-behavioral-and-star.md	STAR stories from YOUR projects, LPs, questions to ask
10	10-mock-interviews.md	4 full timed mock loops with model answers

---

5. A 4-week study plan (≈10 hrs/week)

Week 1 — Coding foundations (the filters)

- **Days 1–2:** Module 01 SQL — work every problem on paper, then verify. Re-do the 5 patterns until automatic.
- **Days 3–4:** Module 02 Python/PySpark — implement each problem yourself before reading the answer.
- **Day 5:** Module 03 Spark internals — shuffles, partitioning, joins, skew, AQE, Delta log.
- **Weekend:** Mock #1 in 10-mock-interviews.md (timed).

Week 2 — Modeling, pipelines, your platform

- **Days 1–2:** Module 04 — SCDs, grain, star vs snowflake, medallion. Draw schemas by hand.
- **Days 3–4:** Module 05 — batch vs streaming, idempotency, CDC, exactly-once, backfills.
- **Day 5:** Module 06 — rehearse your Azure/Fabric stories out loud (this is your edge).
- **Weekend:** Mock #2 (system design focus).

Week 3 — Close the gaps

- **Days 1–2:** Module 07 dbt + Snowflake crash courses. Build the mental model; memorize vocabulary.
- **Day 3:** Module 07 Airflow + Scala. Be able to read code and explain concepts.
- **Days 4–5:** Module 08 system design — do 3 problems end-to-end, out loud, with a whiteboard.
- **Weekend:** Mock #3 (gap-area + system design).

Week 4 — Behavioral + polish

- **Days 1–2:** Module 09 — write and rehearse 10 STAR stories from your real projects.
- **Day 3:** Re-drill every "FAQ" list across modules; flag anything still shaky.
- **Days 4–5:** Re-do all 5 SQL patterns and 5 PySpark patterns cold.
- **Weekend:** Mock #4 (full loop) + review.

---

6. How to use each module

Every module follows the same structure so you can drill efficiently:

1. **Topics to prepare** — the checklist of what you must know.
2. **Frequently asked questions (FAQ)** — short Q&A, the "do you know the term" filter.
3. **Patterns** — the reusable templates interviewers reward.
4. **Mock / coding questions** — interview-level problems **with full answers**.

Study rule: **cover the answer, attempt it, then compare.** Passive reading does not survive contact with a live interviewer.

---

7. Your 60-second positioning statement (memorize)

> "I'm a Senior Data Engineer with ~8 years building cloud analytics platforms, currently at Microsoft through Capgemini on the Finance Analytics & AI platform. I specialize in the Azure data stack — Databricks, Delta Lake, ADF, Synapse, and Microsoft Fabric — designing medallion lakehouse architectures in PySpark and SQL. Recently I cut an executive reporting cycle from 5 days to 4 hours with a Bronze/Silver/Gold pipeline, migrated 25 legacy SSAS models to Fabric semantic models saving $120K a year, and built LLM-powered agentic workflows over our finance data. Before Microsoft I worked across healthcare, insurance, and pharma — including a Medicaid claims platform where anomaly-detection dashboards recovered $8M in improper payments. I'm DP-203 certified and I love turning messy data into systems people trust."

Tighten it, time it to 60 seconds, and make every number ready to defend with a follow-up story (see Module 09).

Module 01 — SQL Mastery

Why this module is #1: SQL appeared in **231 of your 244** DE applications. Every loop has a live SQL round. You will be judged on correctness, clarity, and whether you reach for **window functions** instead of self-joins.

Topics to prepare

- **Joins:** INNER, LEFT/RIGHT/FULL, CROSS, SELF, anti-join (`NOT EXISTS`), semi-join (`EXISTS`).
- **Aggregation:** `GROUP BY`, `HAVING`, `GROUPING SETS`, `ROLLUP`, `CUBE`.
- **Window functions:** `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG` / `LEAD`, `SUM() OVER`, `NTILE`, `FIRST_VALUE` / `LAST_VALUE`, frames (`ROWS BETWEEN ... AND ...`).
- **CTEs & recursion:** `WITH`, recursive CTEs (hierarchies, date spines).
- **Set operations:** `UNION [ALL]`, `INTERSECT`, `EXCEPT`.
- **NULL logic:** three-valued logic, `COALESCE`, `NULLIF`, `IS [NOT] DISTINCT FROM`.
- **Date/time:** truncation, date diffs, generating date spines, sessionization.
- **Performance:** indexes, sargability, predicate/projection pushdown, partition pruning, `EXPLAIN`/query plans, avoiding `SELECT *`, why `WHERE` beats `HAVING` for row filters.
- **Correctness traps:** `COUNT(col)` vs `COUNT(*)`, join fan-out (row explosion), `NOT IN` + NULLs.

The 5 patterns interviewers reward

> Memorize these until you can write them without thinking. ~70% of SQL interview questions are one of these five in disguise.

Pattern 1 — Top-N per group (deduplication)

"Most recent order per customer", "top 3 products per category", "latest record per key".

```sql
WITH ranked AS (
  SELECT *,
         ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts DESC) AS rn
  FROM orders
)
SELECT * FROM ranked WHERE rn = 1;          -- rn ≤ 3 for top-3
```

- `ROW_NUMBER` = exactly one per group (arbitrary tie-break).
- `RANK` = ties share a rank, gaps after (1,1,3).
- `DENSE_RANK` = ties share a rank, no gaps (1,1,2). Use for "top 3 distinct salaries".

Pattern 2 — Running / cumulative totals & moving averages

```sql
SELECT
  account_id, txn_date, amount,
  SUM(amount) OVER (PARTITION BY account_id ORDER BY txn_date
                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_balance,
  AVG(amount) OVER (PARTITION BY account_id ORDER BY txn_date
                    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS moving_avg_7
FROM transactions;
```

- Default frame with `ORDER BY` is `RANGE UNBOUNDED PRECEDING ... CURRENT ROW` — explicitly use `ROWS` to avoid surprises with duplicate sort keys.

Pattern 3 — Period-over-period change (`LAG` / `LEAD`)

"Month-over-month growth", "churned-then-returned", "compare to previous row".

```sql
SELECT
  month,
  revenue,
  LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
  revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change,
  ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
        / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 2) AS mom_pct
FROM monthly_revenue;
```

Pattern 4 — Gaps & islands (consecutive runs)

"Longest streak of active days", "consecutive logins", "contiguous ranges".

```sql
WITH marked AS (
  SELECT user_id, activity_date,
         DATEADD(day,
           -ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY activity_date),
           activity_date) AS grp           -- constant within a consecutive run
  FROM activity
)
SELECT user_id, MIN(activity_date) AS streak_start,
       MAX(activity_date) AS streak_end, COUNT(*) AS streak_len
FROM marked
GROUP BY user_id, grp;
```

Key insight: `date - row_number()` is **constant** across consecutive dates, so it becomes the group key.

Pattern 5 — Sessionization (event gap > threshold = new session)

```sql
WITH lagged AS (
  SELECT user_id, event_ts,
         LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts) AS prev_ts
  FROM events
),
flagged AS (
  SELECT *,
         CASE WHEN prev_ts IS NULL
               OR event_ts > DATEADD(minute, 30, prev_ts)
              THEN 1 ELSE 0 END AS new_session
  FROM lagged
)
SELECT user_id, event_ts,
       SUM(new_session) OVER (PARTITION BY user_id ORDER BY event_ts) AS session_id
FROM flagged;
```

Frequently asked questions (rapid Q&A)

Q: `WHERE` vs `HAVING`? A: `WHERE` filters rows **before** aggregation and can use indexes; `HAVING` filters **after** `GROUP BY` on aggregate results. Filter raw rows in `WHERE` whenever possible.

Q: `RANK` vs `DENSE_RANK` vs `ROW_NUMBER`? A: `ROW_NUMBER` = unique sequential (no ties). `RANK` = ties tie, then skips (1,1,3). `DENSE_RANK` = ties tie, no skip (1,1,2).

Q: `COUNT(*)` vs `COUNT(col)` vs `COUNT(DISTINCT col)`? A: `COUNT(*)` counts rows including NULLs. `COUNT(col)` ignores NULLs in `col`. `COUNT(DISTINCT col)` counts distinct non-NULL values.

Q: Why is `NOT IN` dangerous? A: If the subquery returns any `NULL`, `NOT IN` yields **no rows** (everything becomes UNKNOWN). Use `NOT EXISTS` (anti-join) instead — it's NULL-safe and usually optimizes better.

Q: `UNION` vs `UNION ALL`? A: `UNION` removes duplicates (extra sort/hash — slower). `UNION ALL` keeps all rows. Use `UNION ALL` unless you specifically need dedup.

Q: What makes a predicate "sargable"? A: It can use an index. `WHERE col = @x` is sargable; `WHERE FUNC(col) = @x` (e.g., `YEAR(date)=2025`) is not — rewrite as a range: `date ≥ '2025-01-01' AND date < '2026-01-01'`.

Q: How do you find duplicates? A: `GROUP BY key HAVING COUNT(*) > 1`, or `ROW_NUMBER() OVER (PARTITION BY key ORDER BY …)` and keep `rn > 1` to delete.

Q: Logical order of evaluation? A: `FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT`. This is why you can't reference a `SELECT` alias in `WHERE` but can in `ORDER BY`.

Q: How do you handle join fan-out (row explosion)? A: A 1-to-many join multiplies rows and inflates `SUM`. Pre-aggregate the "many" side in a CTE to the join grain **before** joining, or aggregate with `COUNT(DISTINCT)`.

Q: Difference between a correlated and non-correlated subquery? A: A correlated subquery references the outer query and runs per outer row (logically); a non-correlated one runs once. Optimizers often rewrite both to joins.

Q: How do you generate a date spine with no gaps? A: Recursive CTE or a numbers/calendar table, then `LEFT JOIN` your data so missing days show `0`.

Mock SQL problems (with full answers)

M1 — Second-highest salary per department (and handle ties)

Schema: `employees(emp_id, dept_id, salary)` **Ask:** Return each department's 2nd-highest **distinct** salary.

```sql
WITH d AS (
  SELECT dept_id, salary,
         DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dr
  FROM employees
)
SELECT dept_id, salary AS second_highest_salary
FROM d
WHERE dr = 2
GROUP BY dept_id, salary;
```

Why `DENSE_RANK`: "2nd-highest distinct salary" means ties at the top should not consume rank 2. Follow-up if asked "what if a dept has < 2 distinct salaries?" → it simply returns no row for that dept; use a `LEFT JOIN` against `DISTINCT dept_id` if they want NULLs.

M2 — Customers who bought in Jan but not Feb

Schema: `orders(customer_id, order_date)`

```sql
SELECT DISTINCT customer_id
FROM orders j
WHERE order_date ≥ '2025-01-01' AND order_date < '2025-02-01'
  AND NOT EXISTS (
    SELECT 1 FROM orders f
    WHERE f.customer_id = j.customer_id
      AND f.order_date ≥ '2025-02-01' AND f.order_date < '2025-03-01'
  );
```

Talk track: "I use `NOT EXISTS` rather than `NOT IN` because it's NULL-safe and lets the optimizer use an anti-join. I also use half-open date ranges `[start, end)` to be index-friendly and avoid time-component bugs."

M3 — Median order value per region (no built-in MEDIAN)

```sql
WITH r AS (
  SELECT region, amount,
         ROW_NUMBER() OVER (PARTITION BY region ORDER BY amount) AS rn,
         COUNT(*)    OVER (PARTITION BY region)                  AS cnt
  FROM orders
)
SELECT region, AVG(amount) AS median_amount
FROM r
WHERE rn IN ((cnt + 1) / 2, (cnt + 2) / 2)   -- middle one (odd) or middle two (even)
GROUP BY region;
```

(If the dialect has `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount)`, use that and mention it.)

M4 — Rolling 7-day active users (dedup then window)

Schema: `events(user_id, event_date)`

```sql
WITH daily AS (
  SELECT event_date, COUNT(DISTINCT user_id) AS dau
  FROM events
  GROUP BY event_date
)
SELECT event_date,
       SUM(dau) OVER (ORDER BY event_date
                      ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS rolling_7d_events
FROM daily;
```

Trap to call out: "Note rolling **DISTINCT** users over 7 days isn't a simple window sum because a user can appear on multiple days. For true 7-day unique users I'd self-join each day to the prior 6 (or use an approximate distinct like `APPROX_COUNT_DISTINCT` / HyperLogLog at scale)."

M5 — Funnel conversion: % of users who did A → then B → then C

Schema: `events(user_id, step, event_ts)` where step ∈ {'view','cart','purchase'}

```sql
WITH firsts AS (
  SELECT user_id,
         MIN(CASE WHEN step='view'     THEN event_ts END) AS t_view,
         MIN(CASE WHEN step='cart'     THEN event_ts END) AS t_cart,
         MIN(CASE WHEN step='purchase' THEN event_ts END) AS t_buy
  FROM events
  GROUP BY user_id
)
SELECT
  COUNT(*)                                                AS viewers,
  COUNT(CASE WHEN t_cart > t_view THEN 1 END)             AS added_to_cart,
  COUNT(CASE WHEN t_buy  > t_cart THEN 1 END)             AS purchased,
  ROUND(100.0 * COUNT(CASE WHEN t_buy > t_cart THEN 1 END)
        / NULLIF(COUNT(*),0), 2)                          AS view_to_buy_pct
FROM firsts;
```

Talk track: conditional aggregation collapses the event log to one row per user, enforcing **ordered** steps via timestamp comparison — cleaner and faster than three self-joins.

M6 — Detect and delete duplicate rows, keep the earliest

```sql
WITH d AS (
  SELECT ctid_or_pk,
         ROW_NUMBER() OVER (PARTITION BY natural_key ORDER BY created_at) AS rn
  FROM staging
)
DELETE FROM staging
WHERE pk IN (SELECT pk FROM d WHERE rn > 1);
```

Talk track: partition by the **natural/business key**, order by recency, keep `rn = 1`, delete the rest. In Spark/Delta you'd express the keep-set with `MERGE` or `dropDuplicates` instead.

M7 — Longest active streak per user (gaps & islands)

See Pattern 4 above, then:

```sql
WITH marked AS (
  SELECT user_id, activity_date,
         DATEADD(day, -ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY activity_date),
                 activity_date) AS grp
  FROM (SELECT DISTINCT user_id, activity_date FROM activity) a
)
SELECT user_id, MAX(streak_len) AS longest_streak
FROM (
  SELECT user_id, grp, COUNT(*) AS streak_len
  FROM marked GROUP BY user_id, grp
) s
GROUP BY user_id;
```

Note the `DISTINCT` — duplicate dates would break the offset trick.

Performance talking points (say these, you'll stand out)

- "I check the **plan** first — is it doing a seek or a scan? Where's the most expensive operator?"
- "I make predicates **sargable** and lean on **partition pruning** / **predicate pushdown** in the lake."
- "I watch for **join fan-out** and pre-aggregate to the right grain before joining."
- "On columnar engines (Synapse, Snowflake, BigQuery, Spark SQL) I minimize scanned columns and rows, cluster/partition on common filter keys, and avoid row-by-row UDFs."
- "I prefer set-based logic over cursors/loops; if I see `YEAR(col)=` I rewrite to a range."

Module 02 — Python & PySpark

Why: Python appeared in **207/244** roles; PySpark in **54** explicitly (and implied by every Spark/Databricks role). Expect a Python coding screen *and* PySpark transformation questions.

Part A — Python for Data Engineering

Topics to prepare

- **Core data structures:** `list`, `dict`, `set`, `tuple`, when each is O(1) vs O(n).
- **Comprehensions & generators:** `yield`, lazy iteration, memory efficiency for big files.
- **Iterators/itertools:** `groupby`, `islice`, `chain`, `batched` (3.12+).
- `collections`: `defaultdict`, `Counter`, `deque`, `namedtuple`.
- **Functions:** `*args/**kwargs`, closures, decorators, `functools.lru_cache`, `partial`.
- **Files & formats:** CSV, JSON/NDJSON, Parquet (via `pyarrow` / `pandas`), gzip streaming.
- **Concurrency:** threads vs processes, the **GIL**, `concurrent.futures`, when async helps (I/O).
- **Error handling & retries:** exceptions, context managers (`with`), exponential backoff.
- **Testing/quality:** `pytest`, fixtures, mocking external calls, type hints, `dataclasses`.
- **Pandas basics:** `groupby`, `merge`, `pivot`, vectorization vs `apply`, chunked reads.

FAQ (rapid)

Q: What is the GIL and why does it matter for DE? A: The Global Interpreter Lock lets only one thread execute Python bytecode at a time. CPU-bound work doesn't speed up with threads → use **multiprocessing** or push compute to Spark/SQL. **I/O-bound** work (API calls, file/network) **does** benefit from threads or `asyncio` because the GIL releases during I/O.

Q: `list` vs `generator` for a 50GB file? A: A generator (`yield` / `for line in file`) streams one record at a time — constant memory. A list loads everything → OOM. Prefer generators / chunked reads for large data.

Q: How do you make an API-ingestion job resilient? A: Timeouts, retries with **exponential backoff + jitter**, idempotency keys, paginate with cursors, checkpoint progress, and dead-letter records that fail repeatedly.

Q: `is` vs `==`? A: `==` compares value/equality; `is` compares identity (same object). Use `is` only for `None`/singletons.

Q: Shallow vs deep copy? A: Shallow copies the container but shares nested objects; `copy.deepcopy` clones recursively. Matters when mutating nested config/records.

Q: Why type hints + dataclasses in pipelines? A: Self-documenting schemas, catch errors with `mypy`/Pylance, and `@dataclass`/Pydantic models give you validated, structured records instead of loose dicts.

Python coding problems (with answers)

P1 — Stream a large NDJSON file and aggregate by key (constant memory).

```python
import json
from collections import defaultdict

def revenue_by_country(path: str) -> dict[str, float]:
    totals: dict[str, float] = defaultdict(float)
    with open(path, "r", encoding="utf-8") as f:
        for line in f:                      # generator: one line at a time
            if not line.strip():
                continue
            rec = json.loads(line)
            totals[rec["country"]] += rec.get("amount", 0.0)
    return dict(totals)
```

Talk track: streaming + `defaultdict` = O(1) memory in file size, O(n) time.

P2 — Deduplicate records keeping the latest by timestamp.

```python
def latest_per_key(records: list[dict]) -> list[dict]:
    best: dict[str, dict] = {}
    for r in records:
        k = r["id"]
        if k not in best or r["updated_at"] > best[k]["updated_at"]:
            best[k] = r
    return list(best.values())
```

P3 — Retry with exponential backoff + jitter (real ingestion need).

```python
import random, time
from typing import Callable, TypeVar
T = TypeVar("T")

def with_retry(fn: Callable[[], T], retries: int = 5, base: float = 0.5) -> T:
    for attempt in range(retries):
        try:
            return fn()
        except (TimeoutError, ConnectionError) as e:
            if attempt == retries - 1:
                raise
            sleep = base * (2 ** attempt) + random.uniform(0, 0.25)
            time.sleep(sleep)
    raise RuntimeError("unreachable")
```

P4 — Flatten a nested JSON record into dotted columns.

```python
def flatten(obj: dict, prefix: str = "") -> dict:
    out = {}
    for k, v in obj.items():
        key = f"{prefix}{k}"
        if isinstance(v, dict):
            out.update(flatten(v, prefix=f"{key}."))
        else:
            out[key] = v
    return out
# {"a":{"b":1}} -> {"a.b":1}
```

P5 — Parallelize independent API pulls (I/O-bound → threads).

```python
from concurrent.futures import ThreadPoolExecutor, as_completed

def fetch_all(ids: list[str], fetch_one) -> list[dict]:
    out = []
    with ThreadPoolExecutor(max_workers=16) as ex:
        futures = {ex.submit(fetch_one, i): i for i in ids}
        for fut in as_completed(futures):
            out.append(fut.result())
    return out
```

Talk track: threads help here because each task blocks on network I/O (GIL released). CPU-bound → use `ProcessPoolExecutor` or Spark.

Part B — PySpark

Topics to prepare

- **DataFrame API:** `select`, `withColumn`, `filter`, `groupBy` / `agg`, `join`, `union`, `when/otherwise`.
- **Functions:** `pyspark.sql.functions as F` — `col`, `lit`, `coalesce`, `to_date`, `date_add`, `regexp_replace`, `split`, `explode`, `from_json`, `row_number`, `rank`, `sum().over(Window ...)`.
- **Window specs:** `Window.partitionBy().orderBy().rowsBetween(...)`.
- **Schemas:** `StructType`, explicit schema vs `inferSchema`, handling `corrupt_record`.
- **Joins:** shuffle hash, sort-merge, **broadcast** join, skew handling.
- **Partitioning:** `repartition` vs `coalesce`, `partitionBy` on write, `spark.sql.shuffle.partitions`.
- **Performance:** caching/persist, predicate & projection pushdown, AQE, salting, avoiding UDFs.
- **UDFs:** Python UDF vs **pandas UDF (vectorized)** vs native functions (prefer native).
- **Reading/writing:** Parquet/Delta, partition discovery, `mergeSchema`, write modes.

FAQ (rapid)

Q: `repartition` vs `coalesce`? A: `repartition(n)` does a **full shuffle**, can increase or decrease partitions, balances data. `coalesce(n)` **only decreases** partitions, avoids a full shuffle by merging — cheaper but can skew. Use `coalesce` to reduce small output files; `repartition` to fix skew or increase parallelism.

Q: When does Spark actually run? (lazy evaluation) A: Transformations (`select`, `filter`, `join`) are **lazy** — they build a DAG. An **action** (`count`, `collect`, `write`, `show`) triggers execution. This lets Catalyst optimize the whole plan.

Q: `groupByKey` vs `reduceByKey` (RDD) / why avoid `groupBy().collect_list` carelessly? A: `reduceByKey` combines map-side before shuffle (less data moved); `groupByKey` shuffles everything. In DataFrames, prefer aggregations that Catalyst can optimize over wide `collect_list` that materializes big arrays.

Q: What is a broadcast join and when? A: Spark ships a **small** table to every executor, avoiding a shuffle of the big table. Use when one side fits in memory (default auto-broadcast threshold ~10MB; raise it or hint `F.broadcast(small_df)`).

Q: What causes a shuffle? A: Wide transformations: `join`, `groupBy`/agg, `distinct`, `repartition`, window functions. Shuffles write to disk and move data across the network — the usual performance bottleneck.

Q: How do you handle data skew? A: Diagnose with the Spark UI (one task runs forever). Fixes: **salting** the skewed key, enabling **AQE skew join** (`spark.sql.adaptive.skewJoin.enabled`), broadcasting the small side, or isolating hot keys.

Q: Why avoid Python UDFs? A: They serialize rows to Python, breaking Catalyst optimization and pushdown (slow). Prefer built-in `F.*` functions; if you must, use **pandas/vectorized UDFs** (Arrow) which batch rows.

Q: What is AQE (Adaptive Query Execution)? A: Spark 3+ re-optimizes at runtime using actual stats: coalesces shuffle partitions, switches to broadcast joins, and splits skewed partitions. Generally leave it on.

Q: `cache()`/`persist()` — when? A: When a DataFrame is reused across multiple actions. Caching a once-used DF wastes memory. Unpersist when done. Know storage levels (MEMORY_AND_DISK is the safe default).

Q: `partitionBy` on write vs `repartition`? A: `df.write.partitionBy("dt")` creates **directory partitions on disk** (for pruning at read). `repartition` changes **in-memory** partitioning for the current job. Don't partition a column with huge cardinality (small-files problem).

PySpark coding problems (with answers)

S1 — Top order per customer (window dedup).

```python
from pyspark.sql import functions as F, Window

w = Window.partitionBy("customer_id").orderBy(F.col("order_ts").desc())
latest = (orders
          .withColumn("rn", F.row_number().over(w))
          .filter(F.col("rn") == 1)
          .drop("rn"))
```

S2 — Medallion Silver cleanup: dedup, type-cast, quality flag.

```python
from pyspark.sql import functions as F, Window

w = Window.partitionBy("claim_id").orderBy(F.col("ingest_ts").desc())

silver = (bronze
    .withColumn("amount", F.col("amount").cast("decimal(18,2)"))
    .withColumn("service_date", F.to_date("service_date", "yyyy-MM-dd"))
    .withColumn("is_valid",
        (F.col("amount") > 0) & F.col("service_date").isNotNull())
    .withColumn("rn", F.row_number().over(w))
    .filter(F.col("rn") == 1).drop("rn"))

(silver.write.format("delta").mode("overwrite")
       .option("overwriteSchema", "true")
       .saveAsTable("silver.claims"))
```

Talk track: this is the exact Bronze→Silver pattern from my Microsoft/Assurant work — cast, validate, dedup on the business key by latest ingest, flag quality.

S3 — Incremental upsert with Delta `MERGE` (CDC / idempotent load).

```python
from delta.tables import DeltaTable

tgt = DeltaTable.forName(spark, "silver.customer")
(tgt.alias("t")
   .merge(updates.alias("s"), "t.customer_id = s.customer_id")
   .whenMatchedUpdateAll()
   .whenNotMatchedInsertAll()
   .execute())
```

Talk track: `MERGE` makes the load **idempotent** — re-running the same batch doesn't create duplicates, which is critical for safe retries and backfills.

S4 — Parse a Kafka/Event Hub JSON payload (streaming-ready).

```python
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, TimestampType

schema = StructType([
    StructField("event_id", StringType()),
    StructField("user_id",  StringType()),
    StructField("amount",   DoubleType()),
    StructField("ts",       TimestampType()),
])

parsed = (raw                                  # raw has a binary/string "value" column
    .select(F.from_json(F.col("value").cast("string"), schema).alias("d"))
    .select("d.*")
    .filter(F.col("event_id").isNotNull()))
```

S5 — Fix skew with salting on a hot join key.

```python
from pyspark.sql import functions as F

N = 16
big_salted = big.withColumn("salt", (F.rand() * N).cast("int"))
small_exploded = (small
    .withColumn("salt", F.explode(F.array([F.lit(i) for i in range(N)]))))

joined = big_salted.join(small_exploded,
    on=["join_key", "salt"], how="inner").drop("salt")
```

Talk track: when one key dominates, salting spreads it across N buckets so no single task is overloaded; the small side is replicated N times to match.

S6 — Word/line count the DataFrame way (explain lazy vs action).

```python
from pyspark.sql import functions as F
counts = (df.select(F.explode(F.split(F.col("text"), r"\s+")).alias("word"))
            .filter(F.col("word") ≠ "")
            .groupBy("word").count()
            .orderBy(F.col("count").desc()))
counts.show(20)   # show() is the ACTION that triggers the DAG
```

Pandas ↔ PySpark mental map (say this if asked "you only know pandas?")

Pandas	PySpark
`df[df.x > 0]`	`df.filter(F.col("x") > 0)`
`df.assign(y= ... )`	`df.withColumn("y", ... )`
`df.groupby("k").agg( ... )`	`df.groupBy("k").agg( ... )`
`pd.merge(a,b,on="k")`	`a.join(b, "k")`
in-memory, single node	distributed, lazy, cluster

Module 03 — Spark Internals, Databricks & Delta Lake

Why: Spark (149), Databricks (84), Delta Lake (41), lakehouse (51). For Databricks/MongoDB-style vendors and any Azure Databricks role, **internals** separate seniors from juniors. This is your home turf — be able to go three "why?" levels deep.

Part A — Spark architecture & execution

Topics to prepare

- **Cluster anatomy:** Driver, Executors, Cluster Manager (YARN/K8s/Databricks), Tasks, Cores, Slots.
- **Execution hierarchy:** Job → Stage → Task. **Stage boundary = shuffle.**
- **Catalyst optimizer** (logical → optimized logical → physical plans) and **Tungsten** (whole-stage codegen, off-heap memory).
- **Narrow vs wide transformations** (shuffle or not).
- **Partitioning & parallelism**, `spark.sql.shuffle.partitions` (default 200).
- **Joins:** broadcast hash, shuffle hash, **sort-merge** (default for large), skew handling.
- **Memory model:** execution vs storage memory, spill, OOM causes.
- **AQE** and **DPP** (dynamic partition pruning).
- **File layout:** small-files problem, partitioning, bucketing, compaction.

How a Spark job runs (be able to narrate this)

1. You call an **action** → Spark builds a **DAG** of stages from the lineage.
2. DAG is split into **stages at each shuffle boundary**; each stage = a set of **tasks** (one per partition).
3. The **driver** schedules tasks onto **executor** slots; tasks read their partition, run the stage's pipelined operators, and write shuffle output if there's a downstream stage.
4. **Catalyst** optimizes the logical plan (predicate pushdown, column pruning, join reordering); **Tungsten** generates compact bytecode and manages off-heap memory.

FAQ (rapid)

Q: Narrow vs wide transformation? A: **Narrow** = each output partition depends on one input partition (`map`, `filter`, `select`) — no shuffle. **Wide** = output partition depends on many input partitions (`groupBy`, `join`, `distinct`) — requires a shuffle.

Q: What is a shuffle and why is it expensive? A: Redistributing data across partitions/executors by key. It serializes data, writes shuffle files to local disk, and transfers over the network — disk + network + serialization cost. Minimize wide ops, filter early, broadcast small tables.

Q: Driver vs executor responsibilities? A: **Driver** runs your program, builds the DAG, schedules tasks, holds the `SparkSession`, collects results. **Executors** run tasks and cache data. `collect()` of a huge DF can OOM

the **driver**.

Q: Default join strategy and how to change it? A: **Sort-merge join** for two large tables; **broadcast hash join** when one side is small. Force broadcast with `F.broadcast(df)` or tune `spark.sql.autoBroadcastJoinThreshold`.

Q: Common OOM causes? A: `collect()` / `toPandas()` of big data to driver; skewed keys overloading one executor; huge `groupBy` / `collect_list`; too few partitions; exploding joins. Fixes: more partitions, salting, broadcast, avoid collect, increase memory/cores.

Q: `spark.sql.shuffle.partitions` — why tune it? A: Default 200. Too few → big partitions spill/OOM; too many → tiny tasks + scheduling overhead. AQE auto-coalesces post-shuffle partitions, reducing manual tuning.

Q: How do you debug a slow Spark job? A: Open the **Spark UI** → find the slow **stage** → check task duration distribution (skew?), shuffle read/write size, spill, GC time. Look for one straggler task (skew) or massive shuffle (missing broadcast/filter).

Q: Predicate pushdown & partition pruning? A: Catalyst pushes filters down to the data source so Parquet/Delta only reads matching row groups and **only the partition directories** that satisfy the filter — less I/O. DPP does this dynamically using the dimension side of a join.

Part B — Delta Lake internals

Topics to prepare

- **Transaction log (`_delta_log`):** JSON commits + Parquet checkpoints; **ACID** via optimistic concurrency.
- **Time travel:** query by version / timestamp.
- `MERGE`**/upsert**, `UPDATE`, `DELETE` (vs immutable Parquet).
- **Schema enforcement** & **schema evolution** (`mergeSchema`).
- `OPTIMIZE` (compaction) + **Z-ORDER** (multi-dim clustering) / Liquid Clustering.
- `VACUUM` (retention) and the dangers of short retention.
- **CDF** (Change Data Feed) for downstream incremental consumption.
- **Small-files problem** and auto-optimize / auto-compaction.

How Delta gives you ACID (narrate this)

- Every write creates a new **commit** in `_delta_log` listing which Parquet files are **added** and **removed**. Readers see a consistent snapshot (the set of valid files at a version).
- Concurrency uses **optimistic concurrency control**: a writer reads a snapshot, stages changes, and commits only if no conflicting commit happened; otherwise it retries.
- **Time travel** works because old files aren't deleted immediately — you can read version N or a timestamp. `VACUUM` physically removes files older than the retention window.

FAQ (rapid)

Q: Delta vs Parquet? A: Delta = Parquet **+ transaction log**. That log adds ACID transactions, `MERGE` / `UPDATE` / `DELETE`, time travel, schema enforcement/evolution, and reliable concurrent reads/writes. Plain Parquet has none of those guarantees.

Q: What does `OPTIMIZE` do and when do you Z-ORDER? A: `OPTIMIZE` compacts many small files into right-sized files (better scan throughput). `ZORDER BY (col)` co-locates data with similar values so filters on those columns skip more files (data skipping). Z-ORDER on the columns you filter/join on most.

Q: `VACUUM` risk? A: `VACUUM` deletes files no longer referenced, beyond the retention period (default 7 days). Setting retention too low breaks time travel and can corrupt **concurrent readers/long-running queries**. Don't VACUUM with `RETAIN 0 HOURS` in production.

Q: How does `MERGE` enable idempotency / CDC? A: `MERGE` matches on keys and updates/inserts/deletes accordingly, so replaying the same source batch converges to the same state — safe retries and exactly-once-ish upserts.

Q: Schema enforcement vs evolution? A: Enforcement rejects writes whose schema doesn't match (prevents silent corruption). Evolution (`mergeSchema` / `overwriteSchema`) lets you intentionally add columns. You want enforcement by default, evolution on purpose.

Q: What is Change Data Feed (CDF)? A: Delta can emit row-level changes (insert/update/delete with `_change_type`) so downstream jobs process **only what changed** instead of full reloads — powers incremental Silver→Gold.

Q: Liquid Clustering vs partitioning + Z-ORDER? A: Liquid Clustering is a newer Databricks feature that replaces static `partitionBy` + Z-ORDER with adaptive clustering keys — avoids over/under-partitioning and the small-files trap, and you can change keys without rewriting layout.

Part C — Databricks platform

Topics to prepare

- **Delta Live Tables (DLT):** declarative pipelines, expectations (data quality), auto-orchestration, streaming + batch.
- **Workflows / Jobs:** task orchestration, dependencies, retries, alerts.
- **Unity Catalog:** governance, three-level namespace (`catalog.schema.table`), lineage, access control, audit.
- **Cluster types:** all-purpose vs **job clusters**, **Photon** engine, autoscaling, spot/on-demand.
- **Databricks Asset Bundles (DAB):** code + config as IaC for CI/CD.
- **Autoloader (`cloudFiles`):** incremental file ingestion with schema inference/evolution and exactly-once via checkpointing.

FAQ (rapid)

Q: What is DLT and why use it? A: Delta Live Tables is a **declarative** framework: you define `@dlt.table` transformations and **expectations** (`@dlt.expect`) for quality; DLT manages dependencies, orchestration,

retries, incremental processing, and observability automatically. Less plumbing than hand-built notebooks + Workflows.

Q: What problem does Autoloader solve? A: Efficiently ingesting **new files** from cloud storage without re-listing the whole directory. It tracks processed files (checkpoint), supports **schema inference & evolution**, and gives exactly-once ingestion into Bronze.

Q: Job cluster vs all-purpose cluster? A: **Job clusters** spin up for a scheduled job and terminate after (cheaper, isolated). **All-purpose** clusters are shared/interactive for development. Production jobs should use job clusters.

Q: What does Unity Catalog give you? A: Centralized governance across workspaces — fine-grained access control, **column/row-level security**, automatic **lineage**, audit logs, and a unified `catalog.schema.table` namespace. Replaces the legacy Hive metastore per-workspace model.

Q: What is Photon? A: A vectorized C++ execution engine for Databricks that accelerates SQL and DataFrame workloads, especially scans, joins, and aggregations on Delta. Faster + cheaper for the same query.

Mock deep-dive (full answers)

D1 — "Walk me through what happens when you run `df.groupBy('k').count().show()`."

> The `groupBy().count()` are lazy transformations — Spark just extends the logical plan. `show()` is the action that triggers execution. Catalyst optimizes the plan and Spark splits it into two stages around the shuffle that `groupBy` requires. Stage 1 reads the source partitions and does a **map-side partial aggregation** (combiner) to reduce data. The shuffle repartitions by `k`'s hash so all rows for a key land in the same partition. Stage 2 does the final aggregation per key. With AQE on, Spark coalesces the 200 default shuffle partitions down to what the data actually needs, and would split any skewed partition. `show()` only pulls the first 20 rows back to the driver.

D2 — "Your nightly Silver job suddenly takes 4× longer. How do you diagnose?"

> First, Spark UI → which **stage** regressed. I check the task duration histogram: if one task runs far longer than the rest, it's **skew** (a hot key after a new data source, maybe lots of NULL join keys). I check **shuffle read/write** size for a data-volume jump and **spill** for memory pressure. Common root causes: (1) skew → salt the key or enable AQE skew join; (2) small-files explosion in Bronze → run `OPTIMIZE`/enable auto-compaction; (3) a broadcast that stopped happening because the "small" table grew past the threshold → re-hint or raise the threshold; (4) lost partition pruning because a filter became non-sargable. I'd confirm with the plan (`df.explain()`).

D3 — "How do you design an idempotent, incremental Bronze→Silver→Gold pipeline on Delta?"

Bronze: Autoloader ingests raw files exactly-once with checkpointing, schema evolution on, minimal transformation, append-only with an `ingest_ts`. **Silver:** read Bronze incrementally (CDF or checkpoint), cast types, validate with expectations, dedup on the business key by latest `ingest_ts`, then `MERGE` into the Silver Delta table so re-runs are idempotent. **Gold:** aggregate Silver into star-schema facts/dims or semantic models, again via `MERGE`/overwrite of affected partitions. Orchestrate with DLT or Workflows, add retries, and `OPTIMIZE`/Z-ORDER the heavily-queried Gold tables. Idempotency comes from MERGE keys + checkpoints, so any retry or backfill converges to the same state.

Module 04 — Data Modeling & Warehousing

Why: data warehouse (65), lakehouse (51), data lake (47), dimensional modeling (14), star schema (8), semantic model (26). Modeling rounds are where consulting (Accenture/Capgemini/TCS) and fintech (Robinhood/SoFi/Instacart) interviews are won or lost. Your Kimball + medallion background is a strength — make it crisp.

Topics to prepare

- **Kimball vs Inmon vs Data Vault** — philosophies and when each fits.
- **Star vs snowflake schema**; **fact** and **dimension** tables; **grain**.
- **Fact types:** transaction, periodic snapshot, accumulating snapshot; additive / semi-additive / non-additive measures; **factless facts**.
- **Dimension types:** conformed, role-playing, degenerate, junk, **SCD Types 0–6**.
- **Surrogate keys** vs natural/business keys; late-arriving dimensions; early-arriving facts.
- **Normalization** (1NF–3NF) vs **denormalization** for analytics.
- **Medallion architecture** (Bronze/Silver/Gold) and how it maps to staging/integration/presentation.
- **Lakehouse vs warehouse vs lake**; OLTP vs OLAP.
- **Semantic / metric layer** (Power BI semantic models, dbt metrics, LookML).
- **Partitioning, clustering, and storage layout** for query performance.

The non-negotiables (be able to draw these on a whiteboard)

Star schema

- One central **fact** (measures + foreign keys) surrounded by **dimensions** (descriptive attributes).
- Denormalized dims → fewer joins → fast aggregation, simple for BI tools and self-service.

Grain — declare it FIRST

> "The grain of this fact is **one row per** `<event>` **at** `<level>` — e.g., one row per order line item." Everything (which dims attach, which measures are additive) follows from the grain. Getting grain wrong is the #1 modeling mistake.

Slowly Changing Dimensions (know cold — top FAQ)

Type	Behavior	Use when
Type 0	Never changes (retain original)	Immutable attributes (birth date, original credit score)

Type	Behavior	Use when
Type 1	Overwrite, no history	Corrections; history doesn't matter
Type 2	New row + version (`effective_from/to` , `is_current` , surrogate key)	Need full history (address changes, plan tier)
Type 3	Add "previous value" column	Limited history (current + prior only)
Type 4	History table separate from current	Frequently changing attrs split out
Type 6	1+2+3 hybrid (`current` + row versioning + prior col)	Need current value *and* point-in-time history

Type 2 is the most asked. Be able to write the `MERGE`:

```sql
-- Close the current row when attributes changed, then insert the new version.
MERGE INTO dim_customer t
USING staged_customer s
  ON t.customer_bk = s.customer_bk AND t.is_current = 1
WHEN MATCHED AND (t.plan_tier <> s.plan_tier OR t.address <> s.address) THEN
  UPDATE SET t.is_current = 0, t.effective_to = s.effective_from
-- a second pass / INSERT then adds the new current row with a fresh surrogate key.
```

Frequently asked questions (rapid)

Q: Kimball vs Inmon? A: **Inmon** = top-down: build a normalized (3NF) enterprise data warehouse first, then derive dimensional data marts. **Kimball** = bottom-up: build dimensional **star-schema marts** directly, integrated through **conformed dimensions**. Kimball is faster to value and BI-friendly; Inmon emphasizes a single integrated source of truth. Most modern lakehouses are effectively Kimball stars in the Gold layer.

Q: What is Data Vault and when? A: A modeling style (Hubs = business keys, Links = relationships, Satellites = descriptive/temporal attributes) optimized for auditability, scalability, and many source systems. Common in highly regulated enterprises; you still build Kimball marts on top for consumption.

Q: Star vs snowflake schema? A: **Star** = denormalized dimensions (one hop, fast, simple). **Snowflake** = normalized dimensions into sub-dimensions (saves space, more joins, more complex). Prefer star for analytics; snowflake only when dimensions are huge/shared and storage/consistency matters.

Q: Surrogate key vs natural key — why surrogate? A: A surrogate (system-generated integer/hash) is stable, decouples the warehouse from source-system key changes, enables **SCD Type 2** versioning (same business key, multiple surrogate rows), and joins faster than wide composite natural keys.

Q: Additive vs semi-additive vs non-additive measures? A: **Additive** = sum across all dimensions (revenue). **Semi-additive** = sum across some but not time (account balance, inventory — you average or take last over time). **Non-additive** = ratios/percentages (must recompute from components, never sum).

Q: Transaction vs periodic snapshot vs accumulating snapshot fact? A: **Transaction** = one row per event (most granular). **Periodic snapshot** = one row per entity per period (daily balance). **Accumulating snapshot** = one row per process instance, updated as it moves through milestones (order → ship → deliver), with multiple date FKs.

Q: What is a conformed dimension? A: A dimension (e.g., Date, Customer, Product) shared **consistently** across multiple fact tables/marts so metrics can be compared and "drilled across." The backbone of Kimball enterprise integration.

Q: Degenerate and junk dimensions? A: **Degenerate** = a dimension key with no attributes that lives in the fact (e.g., order number). **Junk** = a dimension that collapses several low-cardinality flags/indicators into one small table to avoid many tiny dimensions.

Q: How does medallion map to classic warehouse layers? A: **Bronze** ≈ raw/landing/staging (as-ingested, append-only). **Silver** ≈ cleansed/conformed integration layer (deduped, typed, validated, business keys). **Gold** ≈ presentation layer (star-schema facts/dims, aggregates, semantic models) for BI/ML.

Q: Lakehouse vs data warehouse vs data lake? A: **Lake** = cheap object storage of raw files, schema-on-read, no guarantees. **Warehouse** = structured, schema-on-write, ACID, fast SQL, but pricey/closed. **Lakehouse** = warehouse capabilities (ACID, governance, performance via Delta/Iceberg) **on** the lake's open files — one copy of data for BI + ML.

Q: OLTP vs OLAP? A: **OLTP** = many small read/write transactions, normalized, row-store, low latency (the source systems). **OLAP** = large analytical scans/aggregations, denormalized/columnar (the warehouse).

Q: What is a semantic / metric layer and why? A: A governed layer that defines business metrics (e.g., "Net Revenue") and relationships **once**, so every dashboard/tool computes them identically. Examples: Power BI semantic models (your DP-203 / Fabric work), dbt metrics, LookML. Prevents "every team has a different revenue number."

Mock modeling problems (with answers)

M1 — "Design a star schema for an e-commerce orders analytics mart."

> **Grain:** one row per **order line item**. **Fact** `fct_order_line`**:** FKs `date_key`, `customer_key`, `product_key`, `store_key`, `promotion_key`; degenerate `order_id`, `line_number`; measures `quantity`, `unit_price`, `discount_amount`, `net_amount` (additive), `is_returned` flag. **Dimensions:** `dim_date` (conformed, role-playing as order_date/ship_date), `dim_customer` (SCD2 for address/segment history), `dim_product` (SCD2 for price/category history), `dim_store`, `dim_promotion`. **Talk track:** declare grain first; keep measures additive at line grain; use surrogate keys; SCD2 on customer/product because history matters for cohort and price analysis.

M2 — "A customer changes their loyalty tier. How do you preserve historical reporting?"

> Model `dim_customer` as **SCD Type 2**: close the current row (`is_current=0`, set `effective_to`), insert a new row with the new tier, fresh surrogate key, `effective_from = change_date`, `is_current=1`. Facts join on the surrogate key valid at the event time, so a Q1 order still reflects the **Q1 tier**, while current dashboards filter `is_current=1`. If they only ever need current + immediately prior, Type 3 is cheaper; if they need current value quickly *and* full history, Type 6.

M3 — "How would you model an order-fulfillment process with multiple milestones?"

> **Accumulating snapshot fact**: one row per order, with multiple date FKs (`order_date_key`, `pick_date_key`, `ship_date_key`, `deliver_date_key`) and lag measures (`days_to_ship`, `days_to_deliver`). The row is **updated** as the order advances. This makes pipeline/SLA analysis trivial without joining many transaction facts.

M4 — "Your BI users complain numbers don't match across dashboards. Root cause + fix?"

> Usually each dashboard re-implements metric logic differently (different filters, different revenue definition, double-counting from join fan-out). Fix: a **single semantic/metric layer** (Power BI semantic model / dbt metrics) defining each measure once over conformed dimensions, plus a data contract on grain. I did this migrating 25 SSAS models to Fabric semantic models — one governed definition, 3× faster queries, and the "which number is right" debates stopped.

Module 05 — Pipelines, Orchestration & Streaming

Why: ETL (159), data pipeline (151), ELT (120), Airflow (112), streaming (69), Kafka (43). Every DE role tests pipeline design thinking: idempotency, scheduling, failure handling, batch vs streaming, CDC. (Airflow is one of your **gap areas** — concepts here, hands-on crash course in Module 07.)

Topics to prepare

- **ETL vs ELT** and why ELT won in the cloud.
- **Batch vs micro-batch vs streaming**; latency/throughput trade-offs.
- **Idempotency, exactly-once vs at-least-once**, dedup, deterministic re-runs.
- **Orchestration:** DAGs, dependencies, scheduling, retries, backfills, SLAs, sensors, alerting.
- **CDC (Change Data Capture):** log-based vs query-based; tools (Debezium, ADF, Fivetran).
- **Incremental loading:** watermarks/high-water marks, CDF, merge/upsert.
- **Streaming:** Kafka/Event Hub, Spark Structured Streaming, windowing, watermarks, checkpointing, late data, **Lambda vs Kappa**.
- **Data quality:** validations, expectations, schema/contract enforcement, dead-letter queues.
- **Observability:** lineage, freshness SLAs, monitoring, alerting, reprocessing.

ETL vs ELT (have a crisp answer)

- **ETL:** transform **before** loading (legacy SSIS/Informatica; transform on a separate engine). Good when the target is expensive/rigid or you must mask/aggregate before landing.
- **ELT:** load raw into a cheap, scalable store/warehouse first, **then transform in-place** with the warehouse/Spark's compute (dbt, Spark SQL, Snowflake). Wins because cloud storage is cheap, compute scales elastically, you keep raw for replay, and transforms are version-controlled SQL.
- Your medallion architecture **is** ELT: land raw (Bronze), transform in Silver/Gold.

Idempotency — the senior signal

> "A pipeline is **idempotent** if re-running it produces the same result without duplicates or corruption." Interviewers probe this constantly because retries/backfills are inevitable.

How you achieve it:

- **MERGE/upsert on a business key** instead of blind append.
- **Overwrite by partition** (`replaceWhere` / dynamic partition overwrite) for the affected window.
- **Deterministic transforms** + **checkpoints** (Autoloader/Structured Streaming) for exactly-once.
- **Idempotency keys** for API sinks; dedup on `event_id`.

Exactly-once vs at-least-once

- **At-least-once:** no message lost, but duplicates possible → you must dedup downstream.
- **Exactly-once (effectively-once):** Structured Streaming achieves this with **checkpointing + idempotent sinks (Delta)** — offsets and results commit atomically so replays don't double-count.
- True end-to-end exactly-once needs an idempotent/transactional sink; otherwise design for at-least-once + dedup.

Orchestration concepts (Airflow vocabulary you must speak)

- **DAG** = Directed Acyclic Graph of tasks with dependencies (`a >> b >> c`).
- **Operator/Task** = a unit of work (Python, SQL, Spark submit, sensor).
- **Schedule** = cron/interval; **execution_date / logical date** = the data interval the run represents (key for backfills).
- **Backfill** = run historical intervals to (re)populate past data.
- **Sensor** = waits for a condition (file arrives, partition ready) — prefer **deferrable/reschedule** sensors to avoid wasting workers.
- **Retries + exponential backoff**, **SLA**, **alerting**, **idempotent tasks** so retries are safe.
- **XCom** = small inter-task message passing (don't pass big data through it — pass pointers/paths).

> In interviews say: "I've orchestrated with **ADF pipelines, Databricks Workflows, Azure DevOps, and GitHub Actions**; the Airflow concepts map 1:1 — DAGs, operators, sensors, backfills, retries — and I'm comfortable picking it up." (See Module 07 for a concrete Airflow DAG.)

CDC (Change Data Capture)

- **Why:** propagate only inserts/updates/deletes from a source DB instead of full reloads.
- **Log-based** (Debezium, Azure SQL CDC, GoldenGate): reads the DB transaction log — low impact, captures deletes, near-real-time. Preferred.
- **Query-based** (high-water-mark on `updated_at`): simpler, but misses hard deletes and needs a reliable change timestamp.
- **Landing pattern:** CDC events → Bronze (append) → `MERGE` into Silver by primary key, applying the latest change per key (handle out-of-order with the change LSN/timestamp).

Streaming essentials

Structured Streaming model

- Treats a stream as an **unbounded table**; each trigger processes a **micro-batch** (or continuous).
- **Checkpointing** stores offsets + state for fault tolerance and exactly-once into Delta.

- **Output modes:** append, update, complete.
- **Triggers:** default (ASAP micro-batch), fixed interval, `availableNow` (drain then stop), continuous (low-latency, limited ops).

Windowing & watermarks

- **Tumbling** (fixed, non-overlapping), **sliding** (overlapping), **session** (gap-based) windows.
- **Event time vs processing time** — always aggregate on **event time** for correctness.
- **Watermark** = "I won't wait for data later than X" → bounds state size and decides when a window is final; late data beyond the watermark is dropped (or sent to a late/dead-letter path).

Lambda vs Kappa

- **Lambda:** separate **batch** (accurate, reprocessable) + **speed/streaming** (low latency) layers, merged at serving. More code, two paths to maintain.
- **Kappa: one streaming** pipeline; reprocess by replaying the log. Simpler; viable when your stream store (Kafka) retains history. Modern lakehouse + Delta blurs the two (same code for batch & stream).

FAQ (rapid)

Q: Kafka core concepts? A: **Topics** split into **partitions** (ordering + parallelism unit); **producers** write, **consumer groups** read with each partition consumed by one consumer in the group; **offsets** track position; **replication factor** for durability. Ordering is guaranteed **within a partition** only.

Q: How do you guarantee ordering for a key in Kafka? A: Use the **key** so all events for that key hash to the same partition (ordering within a partition). Cross-partition global ordering isn't guaranteed by design.

Q: How do you handle late-arriving data in streaming? A: Use **event-time windows + watermarks**; allow lateness up to the watermark to update windows, route data later than the watermark to a **late/dead-letter** sink for separate handling or batch reconciliation.

Q: Event Hub vs Kafka? (you have Event Hub) A: Azure Event Hubs is a managed, partitioned event-ingestion service with a **Kafka-compatible endpoint** — same mental model (partitions, consumer groups, offsets/checkpoints). I've ingested from Event Hub into Structured Streaming; the Kafka API is a small step.

Q: How do you backfill a streaming pipeline? A: Re-read the source from an earlier offset/log retention (Kappa-style replay) or run the equivalent **batch** job over the historical window writing to the same idempotent Delta sink via MERGE/replaceWhere so it converges without duplicates.

Q: How do you enforce data quality in a pipeline? A: Schema enforcement at ingest, **expectations** (DLT `@dlt.expect` / Great Expectations / dbt tests) on nullability, ranges, referential integrity, uniqueness; quarantine failing rows to a dead-letter table; track freshness and row-count anomalies; alert on breach. I built Medallion-layer quality scorecards that moved stakeholder confidence from 68% → 94%.

Mock pipeline problems (with answers)

P1 — "Design a daily pipeline ingesting 500GB of transactions into a reporting warehouse."

> Land files in cloud storage; **Autoloader** ingests new files into **Bronze** (append + `ingest_ts`, schema evolution on, exactly-once via checkpoint). **Silver** job reads incrementally, casts/validates with expectations, dedups on transaction id by latest ingest, `MERGE`s into Silver. **Gold** builds star-schema facts/aggregates via partition overwrite (`replaceWhere` on the date). Orchestrate with Workflows/ADF: dependency Bronze→Silver→Gold, retries with backoff, freshness SLA + alerting, and `OPTIMIZE`/Z-ORDER on Gold. Idempotency from MERGE + checkpoints means any retry/backfill is safe. This mirrors my Microsoft pipeline that took reporting from 5 days to 4 hours on 500GB/day.

P2 — "A nightly job failed at 2am and retried 3×, creating duplicate rows. Prevent it."

> The job was **append-based** (not idempotent). I'd switch the sink to a Delta **MERGE** on the business key (or `replaceWhere` overwrite of the target partition) so re-runs converge. Add a processing checkpoint/high-water mark so it resumes from the last committed offset, and make the transform deterministic. Now 3 retries = same final state, zero duplicates.

P3 — "Source DB has 5M-row tables; full reload each night is too slow. Options?"

> Move to **incremental/CDC**. Best: **log-based CDC** (Debezium/Azure SQL CDC) streaming changes to Bronze, then `MERGE` by primary key into Silver — captures updates *and* deletes in near-real-time. If CDC isn't available, query-based incremental on a reliable `updated_at` high-water mark (but it misses hard deletes — handle with periodic reconciliation). Either way, only changed rows move.

P4 — "Real-time fraud alerts within seconds — batch or streaming, and how?"

> **Streaming.** Kafka/Event Hub ingests transactions; Spark Structured Streaming scores each event against rules/a model using **event-time windows + watermark** for velocity features (e.g., "N txns in 5 min per card"); write alerts to a low-latency sink and the full stream to Delta Bronze for replay/training. Checkpointing gives exactly-once into Delta. For accuracy I'd keep a nightly **batch reconciliation** (Lambda) or a Kappa replay to correct any late/dropped data.

Module 06 — Azure, Synapse, Fabric & Event Hub (Your Differentiator)

Why: Azure (119), ADF (30), Synapse (29), Fabric (24), semantic model (26). This is your **edge** over generic candidates. For Azure Databricks / Microsoft / Capgemini / Quisitive / consulting roles, fluency here is the differentiator. Goal: explain each service, when to use it, and how they fit together — backed by your real projects.

Topics to prepare

- **Azure Data Factory (ADF):** pipelines, activities, datasets, linked services, **Integration Runtime** (Azure/Self-hosted/SSIS-IR), triggers, **Mapping Data Flows** vs orchestration-only, parameterization, ADF vs Synapse pipelines.
- **Azure Synapse Analytics:** dedicated vs **serverless** SQL pools, Spark pools, distribution (hash/round-robin/replicated), PolyBase/COPY, external tables.
- **Microsoft Fabric / OneLake:** Lakehouse vs Warehouse items, **shortcuts**, **Direct Lake**, Dataflows Gen2, Data Pipelines, semantic models, capacity (F-SKUs).
- **Storage:** ADLS Gen2 (hierarchical namespace), containers/medallion zones, lifecycle, security.
- **Streaming ingestion:** Event Hubs (Kafka endpoint), capture to ADLS.
- **Power BI semantic models:** import vs DirectQuery vs **Direct Lake**, DAX, RLS, aggregations.
- **Governance/security:** Microsoft Entra ID, managed identities, **Key Vault**, Purview lineage, Unity Catalog (on Azure Databricks).
- **DevOps:** Azure DevOps pipelines, ARM/Bicep, Databricks Asset Bundles, CI/CD for data.

Service-by-service cheat sheet

Azure Data Factory (ADF)

- **What:** cloud ETL/ELT **orchestration** — move + transform data across 90+ connectors.
- **Building blocks:** *pipeline* (container of activities), *activity* (Copy, Notebook, Stored Proc, Lookup, ForEach, If, Execute Pipeline), *dataset* (shape/location), *linked service* (connection), *Integration Runtime* (compute that runs the work).
- **Integration Runtime types: Azure IR** (cloud, serverless), **Self-hosted IR** (on-prem/private network access), **SSIS IR** (lift-and-shift existing SSIS packages).
- **Mapping Data Flows** = visual, Spark-backed transformations (no-code transform); for heavy transforms many teams instead orchestrate **Databricks notebooks** from ADF.
- **Parameterization** + ForEach = metadata-driven pipelines (one pipeline ingests N tables from a control table).

Azure Synapse Analytics

- **Dedicated SQL pool** = provisioned MPP warehouse; data is sharded across 60 distributions. Choose **distribution**: HASH (large fact, on the join key), ROUND_ROBIN (staging), REPLICATE (small dims).
- **Serverless SQL pool** = pay-per-query SQL directly over files in the lake (great for exploration / logical warehouse, no infra).
- **Spark pool** = managed Spark for transforms/ML.
- **Loading:** `COPY INTO` / PolyBase for high-throughput bulk loads from ADLS.

Microsoft Fabric / OneLake (the newest, learn the vocabulary)

- **OneLake** = one tenant-wide, Delta-based data lake ("OneDrive for data"); every workload reads/writes the same open Delta tables.
- **Shortcuts** = reference data in place (ADLS, S3, another workspace) without copying.
- **Direct Lake** = Power BI reads Delta/Parquet in OneLake **directly** — import-like speed without import refresh or DirectQuery round-trips. Big deal for semantic models.
- **Items:** Lakehouse (Spark + files/tables), Warehouse (T-SQL), Data Pipelines (ADF-like), Dataflows Gen2 (Power Query), semantic models, Notebooks.
- **Capacity:** F-SKUs (compute units) shared across workloads.

Power BI semantic models

- **Storage modes: Import** (data cached in-memory VertiPaq, fast, needs refresh), **DirectQuery** (live queries to source, no copy, slower), **Direct Lake** (Fabric — best of both over OneLake Delta).
- **DAX** measures + **Row-Level Security (RLS)** + aggregation tables for performance.
- This is where your **25 SSAS → Fabric semantic model** migration lives (3× faster, $120K saved).

Frequently asked questions (rapid)

Q: ADF vs Databricks — when each? A: **ADF** for orchestration and connector-rich movement (extract/land, scheduling, control flow). **Databricks** for heavy, code-first transformation (PySpark/SQL on Delta at scale). Common pattern: ADF orchestrates and triggers Databricks notebooks for the transform. (Synapse Pipelines ≈ ADF embedded in Synapse.)

Q: Synapse dedicated vs serverless SQL pool? A: **Dedicated** = provisioned MPP, predictable performance, you pay for reserved compute — for steady high-concurrency warehouses. **Serverless** = on-demand T-SQL over lake files, pay per TB scanned — for ad-hoc/exploration and a logical warehouse without managing infra.

Q: How do you choose a distribution in a Synapse dedicated pool? A: **HASH** on a high-cardinality join/aggregation key for big facts (co-locates joins, avoids data movement); **REPLICATE** for small dimensions (copy to every node); **ROUND_ROBIN** for staging/load where there's no good key yet.

Q: What is Direct Lake and why does it matter? A: A Fabric storage mode where Power BI reads Delta tables in OneLake **directly** in memory — eliminating both import-refresh latency and DirectQuery round-trips. You get import-level performance on near-real-time data with no data duplication.

Q: Import vs DirectQuery in Power BI? A: **Import** caches data in-memory (fastest queries, but stale until refresh, size-limited). **DirectQuery** queries the source live (always fresh, no size limit, slower, source load). Choose by freshness vs performance; Direct Lake (Fabric) gives both.

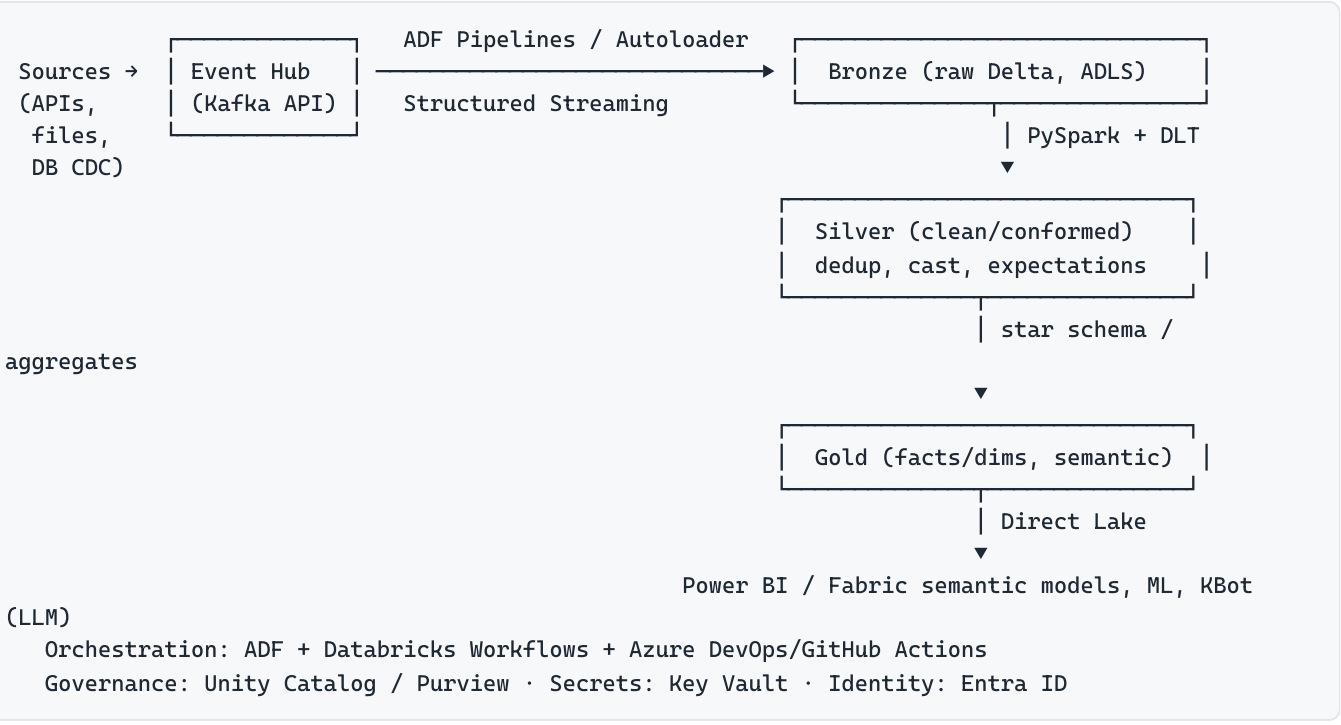
Q: What is OneLake and how is it different from ADLS? A: OneLake is the single, tenant-wide logical lake built on Delta, automatically provisioned with Fabric — all engines share one copy of open data, with **shortcuts** to avoid copies. ADLS Gen2 is the underlying general-purpose hierarchical object store; OneLake sits at a higher, governed, Fabric-native layer.

Q: Self-hosted Integration Runtime — when? A: When ADF must reach data behind a firewall / on-prem / private VNet (e.g., an on-prem SQL Server). You install the SHIR on a machine with network line-of-sight to the source.

Q: How do you secure secrets and identities in Azure data pipelines? A: **Managed identities** for service-to-service auth (no passwords), secrets in **Key Vault** referenced by linked services, **Entra ID** RBAC for access, and Unity Catalog / Purview for data governance and lineage. Never hardcode connection strings.

Q: How do you do CI/CD for Azure data engineering? A: Git-backed ADF/Synapse (publish to adf_publish branch / ARM templates), **Databricks Asset Bundles** for notebooks+jobs as code, deploy via **Azure DevOps**/GitHub Actions across dev/test/prod with parameterized linked services and Key Vault per environment. I automated DataOps this way at Microsoft (Azure DevOps + GitHub Actions + Logic Apps).

Reference architecture you can draw (your actual platform)



Practice: redraw this from memory in 90 seconds and narrate each hop with a metric from your experience (5 days → 4 hours; 500GB/day; 99.97% SOX reconciliation; $120K saved).

Mock platform questions (with answers)

A1 — "Migrate an on-prem SQL Server warehouse to Azure. Walk me through it."

> Assess sources and SLAs first. **Land:** Self-hosted IR in ADF extracts on-prem SQL into **ADLS Bronze** (full first, then CDC/incremental). **Transform:** Databricks PySpark builds Silver (dedup/cast/validate) and Gold star schema on Delta — re-implementing SSIS logic as code. **Serve:** Synapse serverless or Fabric Warehouse for SQL, Power BI semantic models (Direct Lake) for BI. **Cutover:** run parallel, reconcile row counts/control totals, then switch. **Govern:** Key Vault, managed identities, Unity Catalog/Purview lineage, Azure DevOps CI/CD. This is exactly the 8TB on-prem→Synapse pattern I set up at Assurant.

A2 — "When would you NOT use Microsoft Fabric?"

> If the org is standardized on Azure Databricks with mature Unity Catalog governance and complex code-first Spark/ML, Fabric may duplicate capability. Fabric shines for unified BI-centric analytics with Direct Lake and low-ops semantic models; for heavy custom Spark/streaming and multi-cloud, Databricks is often the better core, with Fabric/Power BI as the serving layer. I'd choose based on existing investment, governance needs, and whether the team is SQL/BI- or code-first.

Module 07 — Gap Areas: dbt, Snowflake, Scala & Airflow

Why this is the most important module for your offer rate: your highest *unmet* demand signals are **Scala (172), Airflow (112), dbt (80), and Snowflake** (many dedicated roles). You don't need to be an expert — you need to (1) speak the vocabulary fluently, (2) read the code without panic, and (3) answer "Have you used X?" in a way that converts. Each section below is a crash course + an honest, strong answer script.

> **Golden rule for gaps:** never say a flat "no." Say what you've done that's **equivalent**, show you understand the tool's concepts, and express concrete readiness. Tools are learnable in days; concepts are what they're really testing.

7.1 dbt (data build tool) — demand 80×

The mental model

dbt is the **T in ELT**: you write **SELECT statements as models**; dbt handles dependency ordering, materialization, testing, and docs. It compiles Jinja-templated SQL and runs it **in your warehouse** (Snowflake/BigQuery/Databricks/Synapse). It's basically "software engineering for SQL transforms": version control, modularity, tests, CI/CD.

Core concepts (memorize the vocabulary)

- **Model** = a `.sql` file = one `SELECT` → materialized as a view/table. Files form a DAG via refs.
- **`ref()` / `source()`** = `{{ ref('stg_orders') }}` builds dependencies automatically; `source()` points at raw tables (and powers source freshness checks).
- **Materializations:** `view` (default), `table`, `incremental` (only process new rows), `ephemeral` (inlined CTE). Set via `{{ config(materialized='incremental') }}`.
- **Incremental models:** use `is_incremental()` + a filter on a high-water mark; `unique_key` enables merge/upsert behavior.
- **Tests:** built-in **schema tests** (`unique`, `not_null`, `accepted_values`, `relationships`) in YAML, plus **singular** (custom SQL) and **generic** tests. This is dbt's killer feature for DQ.
- **Snapshots** = dbt's **SCD Type 2** implementation (`strategy: timestamp|check`).
- **Seeds** = small CSVs loaded as tables (lookups).
- **Macros** = reusable Jinja/SQL functions (DRY).
- **Sources + freshness** = declare raw tables and assert data isn't stale.
- **Docs + lineage** = `dbt docs generate` builds a browsable DAG and column-level docs.
- **dbt project layout:** `staging` (1:1 cleaned source) → `intermediate` → `marts` (star schema). This maps **directly** onto your **Bronze → Silver → Gold** medallion mental model.

Minimal example (be able to read/write this)

```sql
-- models/staging/stg_orders.sql
with src as (select * from {{ source('shop', 'orders') }})
select
  order_id,
  customer_id,
  cast(order_ts as timestamp) as order_ts,
  cast(amount as numeric(18,2)) as amount
from src
where amount is not null
```

```sql
-- models/marts/fct_orders.sql
{{ config(materialized='incremental', unique_key='order_id') }}
select * from {{ ref('stg_orders') }}
{% if is_incremental() %}
  where order_ts > (select coalesce(max(order_ts), '1900-01-01') from {{ this }})
{% endif %}
```

```yaml
# models/staging/_staging.yml  — tests as data contracts
models:
  - name: stg_orders
    columns:
      - name: order_id
        tests: [unique, not_null]
      - name: customer_id
        tests:
          - relationships: { to: ref('stg_customers'), field: customer_id }
```

Key commands: `dbt run` (build models), `dbt test` (run tests), `dbt build` (run+test+snapshot+seed in DAG order), `dbt docs generate`.

How to answer "Have you used dbt?"

> "I haven't run dbt in production, but I know it well conceptually and the transition is small for me. My medallion architecture is the same idea — staging→intermediate→marts maps to Bronze→Silver→Gold, and I already do dependency-ordered, tested, version-controlled SQL transforms in Databricks and Azure DevOps. dbt's `ref()` DAG, incremental models with `is_incremental()` and a `unique_key`, schema tests for `unique` / `not_null` / `relationships`, and snapshots for SCD2 all line up with patterns I implement today. I'd be productive in dbt within a week."

7.2 Snowflake — many dedicated roles

The mental model

A cloud data warehouse with **decoupled storage and compute**. Storage is one shared columnar copy; **virtual warehouses** (compute clusters) scale independently and can run concurrently without contention. You pay for

storage + per-second compute.

Architecture (3 layers — say this)

1. **Storage layer** — columnar data in **micro-partitions** (~50–500MB, automatically created), compressed, immutable.
2. **Compute layer** — **virtual warehouses** (T-shirt sizes XS…6XL); resize (scale up) for bigger queries, multi-cluster (scale out) for concurrency. Independent warehouses share the same data.
3. **Cloud services layer** — optimizer, metadata, security, transactions, result cache.

Core concepts (vocabulary)

- **Micro-partitions + pruning:** Snowflake stores min/max metadata per micro-partition and **prunes** partitions that can't match a filter (like data skipping). No manual indexes.
- **Clustering keys:** for very large tables, define a clustering key so related data co-locates → better pruning (analogous to Z-ORDER).
- **Virtual warehouse sizing:** scale **up** (bigger warehouse) for heavy queries; scale **out** (multi-cluster) for many concurrent users; auto-suspend/auto-resume to save cost.
- **Time Travel** (query/restore data up to N days ago) + **Fail-safe** (7-day recovery).
- **Zero-copy cloning:** `CREATE TABLE x CLONE y` — instant, no storage cost until changes (great for dev/test environments).
- **Streams** = CDC on a table (track changes); **Tasks** = scheduled SQL (build pipelines: stream → task → merge).
- **Stages + COPY INTO** = bulk load files; **Snowpipe** = continuous/auto-ingest.
- **Caching:** result cache (identical query), metadata cache, warehouse (local SSD) cache.
- **VARIANT** type + `FLATTEN` for semi-structured JSON.
- **Snowpark** = DataFrame API (Python/Scala/Java) running in Snowflake — close to PySpark ergonomics.

Snowflake vs Databricks/Delta (you'll be asked to compare)

	Snowflake	Databricks lakehouse
Core	SQL data warehouse	Spark + Delta lakehouse
Storage	Proprietary micro-partitions	Open Delta/Parquet in your lake
Compute	Virtual warehouses (SQL)	Clusters (Spark/Photon, SQL)
Strength	Easy SQL, concurrency, low-ops	Big-data eng, ML, streaming, open format
Dedup/upsert	`MERGE`, Streams+Tasks	`MERGE`, CDF

How to answer "Do you know Snowflake?"

> "I've worked primarily in the Azure/Databricks lakehouse, but Snowflake's concepts map cleanly onto what I do. Decoupled storage/compute is like separating ADLS from Databricks clusters; micro-partition pruning is data skipping like Delta's; `MERGE`, Time Travel, and zero-copy cloning have direct Delta analogues; Streams + Tasks are CDC + scheduled transforms like CDF + Workflows. I write the SQL daily, and Snowpark is essentially PySpark. I could be productive immediately and ramp on the cost/warehouse tuning specifics quickly."

7.3 Scala (for Spark) — demand 172×

Reality check

Most "Scala" requirements are "Scala **for Spark**." The Spark DataFrame API is nearly identical across Python and Scala — same methods, different syntax. You won't be asked to write advanced functional Scala for a DE role; you need to **read** it and write basic transformations.

PySpark → Scala Spark (side by side)

SCALA

```
// Scala Spark — same operations you do in PySpark
import org.apache.spark.sql.functions._
import org.apache.spark.sql.expressions.Window

val w = Window.partitionBy("customer_id").orderBy(col("order_ts").desc)

val latest = orders
  .withColumn("rn", row_number().over(w))
  .filter(col("rn") === 1)      // note: === for column equality, not ==
  .drop("rn")

val agg = df.groupBy("country").agg(sum("amount").as("total"))
agg.write.format("delta").mode("overwrite").saveAsTable("gold.country_totals")
```

Syntax differences to remember

- `val` (immutable) / `var` (mutable); types after the name: `val x: Int = 5`.
- Column equality is `===` (and `=!=` for not-equal); functions need `import ... functions._`.
- `.as("alias")` instead of `.alias()`; string interpolation `s"value=$x"`.
- Methods feel the same: `select`, `withColumn`, `filter`, `groupBy`, `join`, `agg`.
- Anonymous functions: `xs.map(x => x * 2)`; `case class` for typed rows/Datasets.

How to answer "Are you comfortable with Scala?"

> "My production Spark is in PySpark, but the DataFrame API is the same engine and nearly the same code — `withColumn`, `groupBy`, window functions, `MERGE`. I can read Scala Spark and write transformations in it; the differences are mostly syntax like `===` for column equality and `val` immutability. For a Scala-heavy codebase I'd ramp within a couple of weeks, and the conceptual Spark knowledge — shuffles, partitioning, joins, skew, Catalyst — carries over completely."

7.4 Airflow — demand 112×

The mental model

Airflow is **orchestration as Python code**: you define a **DAG** of tasks with dependencies; the scheduler runs them on a schedule, handles retries, backfills, and alerting. It does **not** move data itself — it triggers Spark/SQL/containers and tracks state. (Your ADF/Databricks Workflows experience is the same paradigm.)

Core concepts (vocabulary)

- **DAG** = the workflow (Python file) with a `schedule`, `start_date`, dependencies (`a >> b`).
- **Task / Operator** = unit of work: `PythonOperator`, `BashOperator`, `SQLExecuteQueryOperator`, `SparkSubmitOperator`, `DatabricksRunNowOperator`, **TaskFlow** `@task` decorator (modern).
- **Sensor** = waits for a condition (file/partition/external task); prefer **deferrable** sensors.
- **logical/execution date & data interval** = the window a run represents — central to **backfills** and idempotent reruns.
- **XCom** = pass small values between tasks (pass paths, not big data).
- **Retries +** `retry_delay` (exponential backoff), **SLAs**, **pools** (concurrency limits), **catchup** (auto-backfill on/off), **trigger rules** (`all_success`, `one_failed`, …).
- **Connections/Variables** = managed config/secrets (back with a secrets backend, not plaintext).
- **Idempotent tasks** so retries/backfills are safe (same principle you already apply).

Minimal modern DAG (be able to read/explain this)

```python
from airflow.decorators import dag, task
from datetime import datetime

@dag(schedule="0 2 * * *", start_date=datetime(2025, 1, 1),
     catchup=False, default_args={"retries": 3})
def daily_sales():
    @task
    def extract() -> str:
        return "/lake/bronze/sales/2025-01-01"   # returns a pointer via XCom

    @task
    def transform(path: str) -> str:
        # trigger Spark/dbt here; return output location
        return path.replace("bronze", "silver")

    @task
    def load(path: str):
        ...  # MERGE into Gold (idempotent)

    load(transform(extract()))   # dependency graph: extract >> transform >> load

daily_sales()
```

How to answer "Have you used Airflow?"

> "I've orchestrated production pipelines with ADF, Databricks Workflows, and Azure DevOps — same paradigm as Airflow: DAGs of dependent tasks, schedules, retries with backoff, sensors/triggers, backfills over a data interval, and idempotent tasks so reruns are safe. I'm comfortable writing Airflow DAGs with the TaskFlow API and operators like `DatabricksRunNow` and `SparkSubmit`. Because I already design for idempotency and backfills, moving my orchestration to Airflow is a syntax change, not a concept change."

Cheat sheet: mapping YOUR experience → the gap tool

You already do (Azure/Databricks)	Equivalent in the gap tool
Bronze → Silver → Gold	dbt `staging → intermediate → marts`
Delta `MERGE` / CDF	Snowflake `MERGE` / Streams + Tasks; dbt incremental `unique_key`
Data quality scorecards / DLT expectations	dbt `tests` (`unique`, `not_null`, `relationships`)
Delta time travel / Z-ORDER	Snowflake Time Travel / clustering keys
ADF + Databricks Workflows	Airflow DAGs / operators / sensors
PySpark transformations	Scala Spark (same API) / Snowpark
Dev/test data copies	Snowflake zero-copy clone

Use this table in interviews. Translating from your strength to their tool proves transferable mastery far better than claiming hands-on experience you don't have.

Module 08 — Data Engineering System Design

Why: Big tech (Amazon, Microsoft, Dropbox, Reddit, Roblox, Samsara) and platform vendors (Databricks, MongoDB) gate senior offers on a system-design round. They test whether you can take a vague prompt, **ask the right questions**, justify trade-offs, and design something that scales and survives failure. This is also where your architecture experience shines.

The framework (use this every single time)

> Spend the first 5 minutes on **requirements**, not architecture. Most candidates fail by drawing boxes before knowing the problem.

1. Clarify requirements (ask, don't assume)

- **Functional:** What sources? What consumers (BI, ML, API, ops)? What questions must it answer?
- **Scale:** Volume (GB/TB/PB?), velocity (rows/sec, events/day), growth.
- **Latency / freshness:** real-time (seconds), near-real-time (minutes), or daily batch?
- **Consistency / accuracy:** exactly-once? acceptable lag? reprocessing needs?
- **SLAs:** uptime, freshness, query latency.
- **Constraints:** cloud, budget, existing stack, team skills, compliance (PII/HIPAA/SOX).

2. State assumptions & estimate scale

- Back-of-envelope: events/day × bytes = storage/day; peak vs average throughput; retention.

3. High-level architecture (the 5 stages)

```
Ingestion → Storage → Processing/Transform → Serving → Orchestration + Governance
```

4. Drill into components & justify trade-offs

- Batch vs streaming; file format; partitioning; modeling; compute engine; sink.

5. Address the cross-cutting concerns (this is what makes you "senior")

- **Idempotency & exactly-once**, **schema evolution**, **data quality**, **backfills**, **observability/monitoring**, **cost**, **security/governance**, **failure recovery**, **scaling**.

6. Summarize & note future improvements.

The reusable building blocks (your toolbox)

Need	Options (pick + justify)
Ingestion (batch)	ADF/Fivetran/Autoloader, `COPY INTO`

Need	Options (pick + justify)
Ingestion (stream)	Kafka / Event Hub / Kinesis + Structured Streaming
CDC	Debezium / native DB CDC → Bronze → MERGE
Lake storage	ADLS/S3 + **Delta/Iceberg** (ACID, open)
Warehouse	Snowflake / Synapse / BigQuery / Fabric Warehouse
Transform	Spark/Databricks (big data), dbt (SQL-ELT)
Orchestration	Airflow / ADF / Databricks Workflows / Dagster
Serving — BI	Power BI semantic models / Looker
Serving — low latency	Key-value/OLAP store (Redis, Druid, ClickHouse, Cosmos DB)
Quality	DLT expectations / dbt tests / Great Expectations
Governance	Unity Catalog / Purview, lineage, RBAC

Worked example 1 — Design a data warehouse / analytics platform for an e-commerce company

Clarify: sources (orders OLTP DB, web/clickstream events, ad spend API, inventory); consumers (exec dashboards, marketing, finance, DS); freshness (dashboards daily, some near-real-time); scale (~50M events/day, ~5TB historical).

Design:

- **Ingestion:** CDC (Debezium/native) from the orders DB → Bronze; **clickstream via Kafka/Event Hub → Structured Streaming** → Bronze; APIs via daily ADF/Airflow batch.
- **Storage:** ADLS/S3 + **Delta**, medallion zones. Partition events by `event_date`.
- **Transform:** Spark/dbt builds **Silver** (clean, dedup, conform keys) then **Gold** star schemas: `fct_orders` (grain = order line), `fct_web_events`, conformed `dim_customer` (SCD2), `dim_product` (SCD2), `dim_date`.
- **Serving:** Power BI / Snowflake-style warehouse for BI; a feature table for DS.
- **Orchestration:** Airflow/Workflows DAG Bronze→Silver→Gold with retries, freshness SLAs.
- **Cross-cutting:** MERGE-based idempotency; expectations + dbt tests; OPTIMIZE/Z-ORDER hot Gold tables; Unity Catalog governance; monitoring on freshness/row-count anomalies.

Trade-offs to voice: ELT over ETL (cheap storage, replayable raw); Delta for ACID on the lake (one copy for BI+ML) vs a closed warehouse; partitioning by date for pruning but watch small files.

Worked example 2 — Real-time analytics / metrics pipeline (e.g., live dashboard of events)

Clarify: latency target (seconds), metrics (counts, unique users, top-N), accuracy vs speed, volume (100K events/sec peak), late data tolerance.

Design (Kappa-leaning):

- **Ingest:** producers → **Kafka** (partitioned by entity key for ordering + parallelism).
- **Process: Spark Structured Streaming** (or Flink) with **event-time tumbling windows + watermark**; compute windowed aggregates; **checkpointing** for exactly-once into Delta.
- **Serve:** write aggregates to a **low-latency OLAP store** (Druid/ClickHouse/Cosmos DB) for the dashboard; raw stream also lands in Delta Bronze for replay/backfill.
- **Accuracy:** unique counts via HyperLogLog (`approx_count_distinct`) for speed; nightly **batch reconciliation** (Lambda) corrects approximations and late data.

Cross-cutting: handle **late data** beyond watermark via a late/dead-letter path; **backpressure** via Kafka buffering; scale by adding partitions + executors; monitor consumer lag.

Trade-off to voice: Kappa (one streaming path, replay from Kafka) is simpler to maintain; add a batch layer (Lambda) only if you need exact historical correctness the stream can't guarantee.

Worked example 3 — Ingest from 100s of source tables (metadata-driven framework)

Clarify: number of tables, schema drift frequency, full vs incremental, SLAs.

Design:

- A **control/metadata table** lists each source (connection, table, load type, watermark column, keys).
- A **single parameterized pipeline** (ADF ForEach / Airflow dynamic task mapping) iterates the control table: extract → land Bronze → MERGE Silver. No per-table code.
- **Incremental** via high-water mark or CDC; **schema evolution** handled with `mergeSchema`.
- **Quality + lineage** logged per table; failures isolated (one bad table doesn't fail the batch).

Why they love this: it shows you build **scalable frameworks**, not 100 copy-paste pipelines — exactly the metadata-driven pattern senior DEs are expected to know.

Worked example 4 — CDC pipeline from an OLTP database to the lakehouse

Design:

- **Log-based CDC** (Debezium → Kafka, or native DB CDC) captures inserts/updates/**deletes**.
- Land change events append-only in **Bronze** with the change LSN/timestamp and op type.
- **MERGE into Silver** by primary key, applying the **latest** change per key (order by LSN); handle deletes (soft-delete flag or physical delete) and **out-of-order** events.
- Optionally maintain **SCD2 history** in a separate Silver table for point-in-time analysis.

Cross-cutting: idempotent MERGE (replays safe), schema drift handling, initial **snapshot + incremental** bootstrap, monitoring replication lag.

Worked example 5 — Data quality / observability framework

Design layers:

- **Schema/contract** enforcement at ingest (reject or quarantine).
- **Expectations/tests:** nullability, ranges, uniqueness, referential integrity, freshness, row-count vs forecast (anomaly detection). Tools: DLT `expect`, dbt tests, Great Expectations.
- **Quarantine / dead-letter** failing rows; don't fail the whole batch silently.
- **Metrics + alerting:** freshness SLA, volume drift, null-rate spikes → alert + auto-reprocess.
- **Lineage** (Unity Catalog/Purview) so you can trace and impact-analyze.

This is your real story: Medallion-layer **quality scorecards** that took stakeholder confidence from 68% → 94%, plus SOX audit automation at 99.97% accuracy.

Worked example 6 — Batch + streaming unification (Lambda vs Kappa decision)

Prompt: "We need both historical accuracy and real-time freshness."

- **Lambda:** batch layer (accurate, reprocessable Delta) + speed layer (streaming approximations), merged at serving. Cost: two codebases, reconciliation logic.
- **Kappa:** single streaming pipeline; reprocess by replaying Kafka/Delta history. Cost: needs log retention; some heavy historical recompute is awkward.
- **Modern lakehouse answer:** Delta + Structured Streaming lets you run the **same transformation code** for batch and stream (`trigger=availableNow` for batch, continuous for stream), collapsing much of Lambda's duplication. Choose based on accuracy guarantees and team complexity budget.

Cross-cutting checklist (run through these out loud — earns senior signal)

- [ ] **Idempotency:** MERGE/replaceWhere + checkpoints → safe retries/backfills.
- [ ] **Exactly-once vs at-least-once:** stated and justified.
- [ ] **Schema evolution:** how new/changed columns are handled.
- [ ] **Data quality:** validations, quarantine, alerting.
- [ ] **Backfill/reprocessing:** how you rebuild history.
- [ ] **Partitioning & file sizing:** pruning + small-files mitigation (OPTIMIZE).
- [ ] **Scaling:** more partitions/executors; skew handling (salting/AQE).
- [ ] **Observability:** lineage, freshness, monitoring, on-call alerts.
- [ ] **Cost:** storage tiers, job vs all-purpose clusters, auto-suspend, scanned-data reduction.
- [ ] **Security/governance:** PII handling, RBAC, encryption, masking, compliance (HIPAA/SOX/GDPR).
- [ ] **Failure recovery:** retries, checkpoints, replay, DLQ.

Common system-design FAQ

Q: How do you decide batch vs streaming? A: By **freshness requirement vs complexity/cost**. If business value needs sub-minute data (fraud, live ops), stream. If hourly/daily is fine (most reporting), batch is simpler, cheaper, easier to test and backfill. Don't stream by default — justify it.

Q: How do you choose a file format? A: **Columnar (Parquet)** for analytics (compression, column pruning), wrapped in **Delta/Iceberg** for ACID + time travel. Avro/JSON for row-oriented streaming payloads/landing. Never CSV for big analytical storage.

Q: How do you choose partition keys? A: On the column(s) most used in filters/joins (often **date**), with cardinality that yields reasonably sized files (avoid millions of tiny partitions). Combine with clustering/Z-ORDER for secondary filter columns.

Q: How do you handle PII / compliance in the design? A: Classify and tag PII; encrypt at rest/in transit; mask or tokenize; row/column-level security; least -privilege RBAC; audit access (Unity Catalog/Purview); data retention/right-to-be-forgotten via Delta `DELETE` / `MERGE`. I've done GDPR classification + RLS for 8 international markets at Assurant.

Module 09 — Behavioral & STAR (Built From Your Real Projects)

Why: Every loop ends with behavioral. Amazon makes it ~50% of the decision (Leadership Principles), and fintech/consulting all probe impact, conflict, and ownership. You have **genuinely strong, quantified stories** — this module turns your `candidate.json` history into rehearsed answers.

The STAR framework

- **S**ituation — context (1–2 sentences; set the stakes).
- **T**ask — your specific responsibility/goal.
- **A**ction — what **you** did (use "I", show decisions and trade-offs; 50% of the answer).
- **R**esult — quantified outcome + what you learned.

Keep each answer **90 seconds**. Lead with the headline result, then back-fill. Always have a follow-up metric ready.

Your story bank (memorize these 6 — they cover ~90% of questions)

Story A — "Impact / biggest accomplishment" → Microsoft medallion pipeline

- **S:** At Microsoft (via Capgemini) on Finance Analytics, executive reporting took **5 days**; analysts drowned in ad-hoc data requests.
- **T:** I owned the re-architecture of the reporting data platform.
- **A:** Designed a **Bronze/Silver/Gold medallion** pipeline on **Azure Databricks** with **Delta Live Tables** and **structured streaming** from **Event Hub** via ADF; built PySpark transforms for **500GB+/day**, with automated quality validation and observability notebooks.
- **R:** Reporting cycle dropped from **5 days** → **4 hours**, ad-hoc requests fell **60%**, and data freshness went from T+2 to near-real-time. Lesson: declarative pipelines + quality gates beat heroics.

Story B — "Drove measurable business value" → Conduent Medicaid $8M recovery

- **S:** State Medicaid programs were losing money to improper provider payments across **100M+ claims/yr**.
- **T:** Build BI that surfaced anomalies in mission-critical government reporting (99.9% SLA).
- **A:** Architected SQL Server + SSIS Kimball pipelines and **Tableau anomaly-detection dashboards**; optimized partitioning to cut month-end reporting from **18h** → **2h**.
- **R:** Recovered **$8M** in improper payments, influenced policy affecting **2M+ beneficiaries**. Lesson: data quality has direct dollar and human impact.

Story C — "Cost savings / technical migration" → 25 SSAS models → Fabric

- **S:** 25 legacy Analysis Services models were slow and expensive to maintain.

- **T:** Modernize the semantic layer.
- **A:** Migrated them to **Microsoft Fabric semantic models** on Synapse, consolidating into a Lakehouse and standardizing metric definitions.
- **R: 3× query performance**, **$120K/yr** infra savings, and consistent numbers across dashboards. Lesson: a single governed semantic layer kills "whose number is right" debates.

Story D — "Ownership / reliability / compliance" → SOX MUAR automation

- **S:** Quarterly SOX MUAR audits reconciling **2M+ monthly transactions** were manual and risky.
- **T:** Automate the DataOps and audit trail.
- **A:** Built end-to-end pipelines with **Azure DevOps, GitHub Actions, and Logic App** automation, generating audit trails and reconciliations.
- **R: 99.97%** reconciliation accuracy, eliminated **$3M** in audit exposure. Lesson: automate the controls, not just the data.

Story E — "Innovation / learning something new" → Mercury KBot (LLM/agentic)

- **S:** 200+ analysts couldn't easily query enterprise finance data.
- **T:** Enable natural-language access without compromising governance.
- **A:** Built **LLM-integrated agentic workflows** with **LangGraph + Copilot Studio** over **Cosmos DB**-backed finance data.
- **R:** Self-service NL querying for 200+ analysts. Lesson: I proactively learn emerging tech (RAG, agents) and apply it to real data problems.

Story F — "Trust / stakeholder management" → 68% → 94% confidence

- **S:** Stakeholders distrusted platform data (68% confidence score).
- **T:** Rebuild trust in the data.
- **A:** Pioneered **Medallion-layer quality scorecards**, lineage documentation, and automated audit trails so users could see data health and provenance.
- **R:** Confidence rose to **94%**. Lesson: trust is engineered through transparency, not asserted.

---

15 behavioral questions with model answers (mapped to your stories)

1. Tell me about yourself. → Use the 60-second positioning statement in the README.

2. Most challenging technical problem you solved?

> Story A. Emphasize the *decisions*: why DLT over hand-built notebooks, how I handled 500GB/day and ensured idempotency with MERGE + checkpoints, and how quality gates prevented bad data reaching execs.

3. A project with measurable business impact?

> Story B ($8M) or C ($120K). Lead with the number, then the technical "how".

4. Tell me about a time you dealt with poor data quality.

> Story F. "I treat data quality as an engineered property — scorecards, expectations, lineage — not a hope. Moving 68%→94% confidence came from making data health *visible* and quarantining bad rows."

5. A time you had to learn a new technology quickly.

> Story E (LLM/LangGraph) — proves you ramp fast, which directly answers their worry about your **gap areas** (dbt/Snowflake/Airflow/Scala).

6. Tell me about a conflict with a teammate or stakeholder.

> Pick a real disagreement (e.g., a stakeholder wanting a quick hardcoded fix vs. a maintainable model). "I aligned on the goal, showed the maintenance cost of the shortcut with data, proposed a phased approach that delivered a quick win *and* the durable solution, and we shipped both. I disagree with respect and back it with evidence."

7. A time you failed or made a mistake.

> Be honest and show learning. Example shape: "Early on I shipped a pipeline that wasn't idempotent; a 2am retry created duplicates in a report. I owned it, fixed it with a MERGE-based load and checkpoints, and added a reconciliation check. Now idempotency is the *first* thing I design for." (Never a non-failure humblebrag; show real ownership + the systemic fix.)

8. How do you prioritize when everything is urgent?

> "I rank by business impact and risk, clarify with stakeholders, and protect critical SLAs first. During month-end I sequence so the 99.9% SLA government reports never slip, and defer nice-to-haves." (Story B)

9. A time you improved performance/efficiency.

> 18h→2h (Story B) or 5 days→4 hours (Story A). Walk through the *diagnosis* (Spark UI / query plan / partitioning) then the fix.

10. How do you handle ambiguous requirements?

> "I start with the consumer and the question they're answering, define the grain and SLAs, make assumptions explicit, and ship an iterative slice for feedback rather than guessing for months."

11. A time you influenced without authority.

> Story C/F — standardizing the semantic layer required convincing multiple teams to adopt one metric definition. "I built the case with performance numbers and reduced-maintenance evidence, piloted with one team, then let the results sell it."

12. Tell me about mentoring or leading.

> If you've guided juniors/contractors: code reviews, pairing on PySpark, establishing the medallion + quality standards others now follow. Frame as raising the team's bar.

13. How do you ensure your pipelines are reliable?

> Idempotency (MERGE/checkpoints), expectations/tests, monitoring + freshness SLAs, retries with backoff, lineage, and on-call runbooks. (Story D)

14. Why do you want to work here / leave your current role?

> Tie their mission/stack to your trajectory. Be positive about Microsoft/Capgemini; frame the move as wanting bigger scale / ownership / a specific tech direction (e.g., "deepen on streaming/lakehouse at product scale"). Never bad-mouth.

15. Where do you see yourself / how do you grow?

> "Toward staff/lead data engineering — owning platform architecture and mentoring — while staying hands-on. I deliberately keep learning emerging areas like LLM/agentic data apps."

Amazon Leadership Principles (you applied to Amazon — prep these specifically)

Map your stories to the LPs they probe most:

Leadership Principle	Your story
Customer Obsession	Story B — recovered $8M; 2M+ beneficiaries served reliably
Ownership	Story D — owned SOX automation end-to-end; eliminated $3M exposure
Invent and Simplify	Story A / E — DLT medallion; LLM KBot for self-service
Dive Deep	Spark UI debugging (Module 03 D2); partitioning 18h→2h
Deliver Results	5 days→4 hours; $120K saved; 99.97% accuracy
Bias for Action	Iterative slice delivery under ambiguity

Leadership Principle	Your story
Earn Trust	Story F — 68%→94% confidence via transparency
Are Right, A Lot	Architecture choices that scaled (Delta, medallion)
Learn and Be Curious	LangGraph/RAG; readiness on dbt/Snowflake/Scala/Airflow

For Amazon: prepare **2 distinct stories per LP**, all in STAR, all with metrics, and expect deep "why/what-if" follow-ups ("Dive Deep" means they will drill 4 levels down — know your numbers).

Sponsorship / logistics scripts (you're on H-1B requiring transfer)

Have these ready, calm and factual:

> **"What's your work authorization?"** → "I'm currently on H-1B and would need a standard H-1B transfer; I'm authorized to work and have been employed continuously in the US. No cap issues since I've been counted. Happy to connect your immigration team with mine to make it smooth."

Know your numbers: target comp (~$150K, negotiable per `candidate.json`), start (~2 weeks), remote/hybrid preference, and openness to US relocation. Decide your floor before the call so you don't negotiate live.

Smart questions to ask THEM (always have 4–5)

- "What does the data platform look like today, and what's the biggest pain you'd want this role to fix?"
- "Batch, streaming, or both — and what are your freshness SLAs?"
- "How do you handle data quality and on-call for pipelines?"
- "What's the split between building new platform vs. maintaining existing?"
- "How do teams consume the data — BI, ML, reverse-ETL — and who owns the semantic layer?"
- "What does success in the first 6 months look like?"

Asking sharp, design-level questions signals seniority as much as your answers do.

Module 10 — Full Mock Interviews (Timed, With Model Answers)

How to use: Set a timer. **Cover the answers.** Speak (or write) your response out loud as if a human is listening, *then* compare. Record yourself for the behavioral ones. Each mock simulates a real loop drawn from the company archetypes in your applications.

- **Mock 1 — Fintech / high-growth** (Robinhood / SoFi / Instacart / Brex style)
- **Mock 2 — Big Tech** (Amazon / Microsoft / Dropbox style)
- **Mock 3 — Platform vendor** (Databricks / MongoDB style — deep internals)
- **Mock 4 — Consulting / Azure shop** (Accenture / Capgemini / Quisitive style)

MOCK 1 — Fintech Senior Data/Analytics Engineer (60 min)

Round 1.1 — SQL (15 min)

Q: `transactions(user_id, txn_ts, amount, status)`. For each user, find their **first** successful transaction and the **days until** their second successful transaction.

▾ **Model answer**

SQL

```
WITH succ AS (
  SELECT user_id, txn_ts,
         ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY txn_ts) AS rn
  FROM transactions
  WHERE status = 'success'
),
p AS (
  SELECT user_id,
         MAX(CASE WHEN rn = 1 THEN txn_ts END) AS first_ts,
         MAX(CASE WHEN rn = 2 THEN txn_ts END) AS second_ts
  FROM succ
  WHERE rn ≤ 2
  GROUP BY user_id
)
SELECT user_id, first_ts, second_ts,
       DATEDIFF(day, first_ts, second_ts) AS days_to_second
FROM p
WHERE second_ts IS NOT NULL;
```

Talk track: filter to success first, rank, pivot the first two with conditional aggregation, then diff. Mention users with only one success return NULL (filtered out, or kept with a LEFT pattern if needed).

Round 1.2 — Analytics Engineering / dbt (10 min)

Q: "How would you build a trustworthy daily revenue metric used by 5 teams?"

▾ **Model answer**

Define it **once** in a semantic/metric layer (dbt metric or Power BI measure) over a conformed `fct_revenue` at a declared grain. In dbt: a `staging` model per source, an `intermediate` join/clean layer, a `marts/fct_revenue` incremental model (`unique_key`), plus **tests** (`not_null`, `unique`, `relationships`) and a **source freshness** check. CI runs `dbt build` on PRs. Now every team `ref`s the same model — no divergent definitions. (Tie to your 25-SSAS→Fabric semantic-layer consolidation.)

Round 1.3 — Pipeline design (20 min)

Q: "Design a pipeline to ingest payment events and power both a real-time fraud check and daily finance reporting."

▾ **Model answer**

This is a **Lambda/lakehouse** problem. Stream payment events into **Kafka**; **Structured Streaming** scores fraud in real time with event-time windows + watermark for velocity features, writing alerts to a low-latency store and raw events to **Delta Bronze** (checkpointed, exactly-once). A **batch** Silver/ Gold job builds the finance star schema daily via MERGE (idempotent), reconciling any late data. Same Delta tables serve BI (Power BI/warehouse). Cross-cutting: idempotency, schema evolution, dbt/DLT tests, freshness SLAs, PII masking (it's payments — compliance matters), monitoring consumer lag. Trade-off: streaming only where seconds matter (fraud); batch for reporting to keep it simple and cheap.

Round 1.4 — Behavioral (15 min)

Q: "Tell me about a time you improved data trust." → **Story F** (68%→94%). 90 seconds, STAR, quantified. Follow-up "how did you measure confidence?" → stakeholder survey + reduced data-dispute tickets + freshness/quality scorecards.

MOCK 2 — Big Tech Senior Data Engineer (75 min)

Round 2.1 — Python/PySpark coding (20 min)

Q: Given a DataFrame of `(user_id, page, event_ts)`, compute each user's **session count** where a new session starts after 30 minutes of inactivity.

▾ **Model answer**

PYTHON

```python
from pyspark.sql import functions as F, Window
w = Window.partitionBy("user_id").orderBy("event_ts")

sessions = (events
  .withColumn("prev_ts", F.lag("event_ts").over(w))
  .withColumn("new_session",
      F.when(F.col("prev_ts").isNull() |
             (F.col("event_ts").cast("long") - F.col("prev_ts").cast("long") > 30*60), 1)
       .otherwise(0))
  .withColumn("session_id", F.sum("new_session").over(w))
  .groupBy("user_id").agg(F.max("session_id").alias("session_count")))
```

Talk track: this is the **sessionization** pattern (Module 01 P5) in PySpark — `lag` for the gap, a running `sum` of session-start flags assigns session ids, then aggregate. Note window functions cause a shuffle; partition by `user_id`. Mention skew if a few power users dominate → still fine since each user's data fits a partition, but a single mega-user could be a hot key.

Round 2.2 — Data modeling (15 min)

Q: "Model a ride-sharing trips warehouse for analytics."

▾ **Model answer**

Grain = **one row per completed trip**. `fct_trip`: FKs `date_key`, `rider_key`, `driver_key`, `pickup_loc_key`, `dropoff_loc_key`, `vehicle_key`, `promo_key`; degenerate `trip_id`; measures `distance`, `duration_sec`, `fare`, `tip`, `surge_multiplier` (non-additive — keep components), `driver_payout`. Dims: `dim_rider` / `dim_driver` (SCD2 for rating/tier history), `dim_location`, `dim_date` (role-playing pickup/dropoff), `dim_vehicle`. For lifecycle (requested→matched→completed), an **accumulating snapshot** with milestone date FKs and lag measures. Call out additive vs non-additive (surge), and conformed `dim_date` / `dim_location` reused across trips, payments, incidents facts.

Round 2.3 — System design (25 min)

Q: "Design the analytics data platform for a product with 100M daily events."

▾ **Model answer**

Use the Module 08 framework. Clarify consumers/freshness/scale (100M/day ≈ ~1.2K eps avg, peak higher; sized storage ≈ events × bytes). Ingest: clickstream via Kafka/Event Hub → Structured Streaming → **Delta Bronze**; OLTP via **CDC**; APIs via batch. Medallion transform in Spark/dbt to conformed Gold stars. Serve BI via semantic models, DS via feature tables, low-latency metrics via an OLAP store. Orchestrate with Airflow/Workflows. Then **spend half the time on cross-cutting**: idempotency (MERGE + checkpoints), schema evolution, DQ expectations + quarantine, partitioning by date + OPTIMIZE/Z-ORDER, skew handling (salting/AQE), backfills via replay, observability (lineage, freshness, consumer lag), cost (job clusters, auto-suspend, scanned-data reduction), governance (Unity Catalog, PII). Close with trade-offs and v2 improvements (e.g., move from Lambda to unified Delta streaming).

Round 2.4 — Behavioral / LP deep dive (15 min)

Q (Dive Deep): "Tell me about the hardest bug/perf issue you debugged." → Module 03 **D2** (4× slow Silver job). Expect 4 levels of "why?": be ready with the exact Spark UI signals (task skew, shuffle size, spill), the root cause (hot key / small files / lost broadcast), and the fix (salt / OPTIMIZE / re-broadcast). **Know your numbers.**

MOCK 3 — Data Platform Vendor (Databricks / MongoDB) (60 min)

Round 3.1 — Spark internals rapid-fire (15 min)

▾ **Q&A**

"Job → stage → task — what defines a stage boundary?" A shuffle (wide dependency).

"Default join for two big tables?" Sort-merge; broadcast if one side is small.

"How does AQE help?" Runtime re-optimization: coalesce shuffle partitions, switch to broadcast, split skewed partitions.

"What causes an OOM on the driver?" `collect()` / `toPandas()` of large data; huge broadcast.

"narrow vs wide?" narrow = 1 input→1 output partition (no shuffle); wide = many→one (shuffle).

"How do you fix skew?" Salting, AQE skew join, broadcast small side, isolate hot keys.

Round 3.2 — Delta deep dive (15 min)

▾ **Q&A**

"How does Delta give ACID?" Transaction log (`_delta_log`) of add/remove file actions + optimistic concurrency; readers see a consistent snapshot.

"OPTIMIZE vs Z-ORDER?" Compaction vs multi-dimensional clustering for data skipping.

"VACUUM danger?" Deletes unreferenced files past retention; too-low retention breaks time travel & concurrent readers.

"How does MERGE help CDC?" Idempotent upsert by key → safe replays, applies latest change.

"CDF?" Row-level change feed for incremental downstream consumption.

"Schema enforcement vs evolution?" Reject mismatches by default; opt-in `mergeSchema` to add cols.

Round 3.3 — Customer-facing scenario (15 min)

Q: "A customer says their Databricks job is slow and expensive. How do you help?"

▾ **Model answer**

Diagnose before prescribing. Pull the **Spark UI / query profile**: look for skew (straggler tasks), excessive shuffle, **small files** in Bronze, spill/GC, and whether they're on an oversized all-purpose cluster. Quick wins: enable **AQE + Photon**, switch prod jobs to **job clusters** with autoscaling and auto-termination, `OPTIMIZE`/Z-ORDER hot tables, partition/prune correctly, broadcast small dims, cache only reused DataFrames, avoid Python UDFs. Then right-size the cluster and use spot for non-critical. Frame it as cost *and* performance — exactly what a platform SA does. Confirm with before/after metrics.

Round 3.4 — Communication (15 min)

Q: "Explain the lakehouse to a VP who only knows data warehouses."

▾ **Model answer**

"A warehouse is great for fast SQL but locks data in a proprietary, pricey system, and you keep a second copy in a lake for ML. A **lakehouse** puts warehouse capabilities — ACID transactions, governance, fast SQL — directly on cheap, open cloud storage using Delta. So you keep **one** copy of data that serves both BI and ML, with no vendor lock-in, lower cost, and real-time + batch on the same tables. You get the reliability of a warehouse with the scale and openness of a lake." (Analogy-driven, no jargon — vendor SAs are scored on this.)

MOCK 4 — Consulting / Azure Data Engineer (45 min)

Round 4.1 — Screening Q&A (10 min)

▾ **Rapid answers**

"Years in data engineering?" ~8 years across finance, healthcare, insurance, pharma.

"ADF vs Databricks?" ADF orchestrates + moves; Databricks transforms at scale; ADF triggers Databricks notebooks.

"Synapse dedicated vs serverless?" Provisioned MPP vs on-demand SQL over lake files.

"DP-203?" Yes — Azure Data Engineer Associate.

"Sponsorship?" H-1B transfer (see Module 09 script).

"Comfortable client-facing?" Yes — I've presented platforms and trained 150+ business users.

Round 4.2 — Azure scenario (20 min)

Q: "A client wants to move their on-prem SQL Server reporting to Azure with minimal disruption."

▾ **Model answer**

Module 06 **A1**: Self-hosted IR in ADF extracts on-prem SQL → ADLS **Bronze** (full then CDC); Databricks PySpark builds Silver/Gold Delta star schema (re-implementing SSIS as code); serve via Synapse serverless / Fabric Warehouse + Power BI semantic models (Direct Lake). Run **parallel**, reconcile control totals, then cut over. Govern with Key Vault, managed identities, Purview lineage; CI/CD via Azure DevOps. Minimal disruption = parallel run + phased cutover + reconciliation. (Tie to my Assurant 8TB on-prem→Synapse migration.)

Round 4.3 — Behavioral (15 min)

Q: "Tell me about working with a difficult stakeholder / changing requirements." → Use the conflict story (Module 09 Q6) or ambiguity story (Q10). Emphasize: align on the goal, make trade-offs visible with data, deliver an iterative slice, keep the relationship intact.

Self-scoring rubric (grade each mock 1–5)

Dimension	1 (weak)	5 (strong)
Correctness	Wrong/buggy	Correct, handles edge cases
Clarity	Rambling	Structured, narrates thinking
Trade-offs	States one option	Compares options + justifies
Seniority signals	Only happy path	Idempotency, scale, failure, cost, governance
Metrics in stories	Vague	Quantified + defensible follow-ups
Communication	Jargon dump	Tailored to audience

Target before real interviews: consistent 4–5 across all dimensions on Mocks 1–3, and a clean, confident sponsorship + comp conversation.

Final-week checklist

- [ ] Re-do all 5 SQL patterns (Module 01) and 6 PySpark problems (Module 02) **cold**.
- [ ] Narrate the Spark job lifecycle + Delta ACID (Module 03) from memory.
- [ ] Draw your reference architecture (Module 06) in 90 seconds.
- [ ] Deliver all 6 STAR stories (Module 09) in ≤90s each, with numbers.
- [ ] Give the honest-but-strong answer for **each** gap: dbt, Snowflake, Scala, Airflow (Module 07).
- [ ] Run one full mock out loud the day before — then rest.

You've got the experience. This course just makes sure the interviewer **sees** it. Good luck.

Appendix A — Gap Analysis: What's Missing & How to Make This a Solid Interview-Prep Pack

This appendix does two jobs: (1) an honest **audit of what the current 10 modules do not yet cover**, ranked by interview risk, and (2) a concrete **roadmap for turning this study material into a publication-grade, printable interview-prep PDF**. The consolidated PDF you are reading already applies several of these improvements — they are marked ✅ **Done** below.

A.1 Technical coverage gaps (ranked by interview risk)

The course is deliberately Azure/Databricks-centric because that is the candidate's strength. That focus is correct, but the following topics appear frequently in job descriptions and live loops and are **not yet covered in depth**. Close them in priority order.

#	Missing topic	Why it matters in interviews	Suggested home	Priority
1	**AWS data stack** — Redshift, Glue, EMR, S3, Athena, Kinesis, Lambda, Step Functions	A large share of DE roles are AWS-first; Azure-only answers lose points	New Module 6B	**High**
2	**GCP data stack** — BigQuery, Dataflow, Dataproc, Pub/Sub, Composer	Common at ad-tech, retail, and Google-adjacent shops	New Module 6C	**High**
3	**Apache Kafka in depth** — partitions, ISR, exactly-once, Kafka Streams, Connect, schema registry	Demand 43×; course only covers it adjacently via Event Hub	Extend Module 05	**High**
4	**Open table formats beyond Delta** — Apache Iceberg & Hudi (hidden partitioning, copy-on-write vs merge-on-read, catalogs)	Iceberg is now asked alongside Delta at senior level	Extend Module 03	**High**
5	**Data Structures & Algorithms screen** — arrays, hash maps, two-pointers, heaps, recursion, Big-O	Big-tech phone screens still gate on a non-SQL coding round	New Module 2B	**High**
6	**Security, privacy & compliance** — PII classification, masking/tokenization, encryption, RBAC/ABAC, GDPR/CCPA/HIPAA/SOX, row/column security	Probed heavily in fintech, health, and government roles	New Module 11	**High**
7	**Data quality & observability tooling** — Great Expectations, Soda, Monte Carlo, dbt tests in depth, data contracts	"How do you guarantee quality?" is a near-universal question	Extend Modules 05/08	Medium
8	**File formats & serialization** — Parquet/ORC/Avro internals, encoding, compression, schema registry/evolution	Frequent "why Parquet?" and "how does columnar work?" follow-ups	Extend Module 08	Medium
9	**Testing & CI/CD for data** — unit/integration tests, data-diff, environment promotion, blue/green, contract tests	Senior signal; shows engineering maturity	New Module 12	Medium

#	Missing topic	Why it matters in interviews	Suggested home	Priority
10	**Cost & performance / FinOps** — cluster sizing, spot, autoscaling, storage tiering, query/scan cost	Increasingly asked as cloud bills grow	Extend Modules 03/08	Medium
11	**Streaming depth** — Flink vs Spark, stateful processing, watermark tuning, end-to-end exactly-once internals	Differentiator for streaming-heavy roles	Extend Module 05	Medium
12	**MLOps / feature engineering adjacency** — feature stores, online vs batch features, vector DBs, RAG/agentic	Ties directly to the candidate's KBot/LLM work	New Module 13	Medium
13	**Orchestration alternatives** — Dagster, Prefect, Mage, asset-based orchestration	Course covers Airflow only; some shops use Dagster/Prefect	Extend Module 07	Low
14	**Behavioral breadth** — 10+ stories, deeper failure/conflict examples, company-specific value maps (Microsoft, Google, Meta)	Amazon-style loops demand 2 stories per principle	Extend Module 09	Low
15	**System-design variety** — multi-region, GDPR-delete, schema-registry-driven, capacity-math worksheets	More reps across problem archetypes builds fluency	Extend Module 08	Low

---

A.2 Structural gaps that weaken it as a *document*

These are about the artifact itself rather than the content. The consolidated PDF fixes the items marked ✅ **Done**; the rest are recommended next steps.

- ✅ **Cover page, table of contents, page numbers, and per-chapter page breaks** (added in this PDF).
- ✅ **Consistent styling** for tables, callouts, and monospaced code with language labels.
- ✅ **This gap-analysis appendix, a one-page cheat sheet, and a glossary** (sections A.4–A.6).
- ⬜ **Master Question Bank** — every question from all modules in one place (no answers) for self-testing, plus a separate answer key.
- ⬜ **Printable flashcards** (question → answer) and a **spaced-repetition schedule**.
- ⬜ **TOC with printed page numbers, running chapter headers, and an auto-generated index** — requires a paged-media engine (see A.3).
- ⬜ **Fillable trackers** — a mock-interview score log and a checkbox-based 4-week calendar.
- ⬜ **Per-company one-pagers** — stack, values, recent news, and likely questions for each target employer.

A.3 How this PDF is built (and how to make it even stronger)

Current pipeline (reliable, zero network): the build script reads the README and all modules, converts Markdown to HTML with `marked`, applies a print stylesheet, and renders to PDF with **Playwright's headless Chromium** (`page.pdf()`). This produces a clean, professional, fully offline document with a clickable TOC and footer page numbers.

Upgrade path for a publication-grade result:

1. **Add Paged.js** to the same Playwright pipeline. It resolves `target-counter()` so the TOC can show **real page numbers**, adds **running chapter headers**, and can generate a back-of-book **index** — the difference between "nice" and "book-quality."
2. **Add syntax highlighting** by vendoring the `highlight.js` browser bundle and running `hljs.highlightAll()` inside the page before `page.pdf()`. Code listings gain color without any runtime network dependency.
3. **Embed a licensed font** (e.g., a clean serif for headings + a programming font for code) so the PDF looks identical on every machine, independent of installed system fonts.
4. **Generate a Master Question Bank automatically** by scanning the Markdown for question markers (`**Q:`, `### M`, `### Round`, `**P`, numbered behavioral prompts) and emitting a deduplicated list.

A.4 One-page cheat sheet (quick reference)

SQL — the 5 patterns: Top-N per group (`ROW_NUMBER`), running/cumulative total (`SUM() OVER`), period-over-period (`LAG`/`LEAD`), gaps & islands (`date - ROW_NUMBER()`), sessionization (gap > threshold → new session). Prefer `NOT EXISTS` over `NOT IN`; filter in `WHERE` not `HAVING`; keep predicates sargable.

Python/PySpark: `repartition` (full shuffle) vs `coalesce` (merge only); **broadcast** the small side; fix skew with **salting** or **AQE**; make loads idempotent with `MERGE`; avoid Python UDFs (use `F.*` or vectorized/pandas UDFs); generators for big files.

Spark/Delta: stage boundary = **shuffle**; sort-merge join default, broadcast when small; `_delta_log` + optimistic concurrency = **ACID**; `OPTIMIZE` (compaction) + `ZORDER` (data skipping); `VACUUM` with care; **CDF** for incremental downstream; **MERGE** for idempotent CDC.

Modeling: declare the grain first; star over snowflake; **SCD Type 2** via close-row + insert `MERGE`; conformed dimensions; additive / semi-additive / non-additive measures; medallion ≈ staging/integration/presentation.

Pipelines: idempotency = `MERGE`/`replaceWhere` + checkpoints; exactly-once = checkpoint + idempotent sink; **log-based CDC** captures deletes; event-time **windows + watermarks** for streaming; Lambda vs Kappa.

Azure: ADF **orchestrates**, Databricks **transforms**; Synapse dedicated (MPP) vs serverless (pay-per-scan); **Direct Lake** = import speed on live OneLake Delta; secrets in **Key Vault** + managed identities.

Gap-area scripts: never say a flat "no" — map your strength to their tool (medallion → dbt staging/marts; Delta MERGE/CDF → Snowflake MERGE/Streams+Tasks; ADF/Workflows → Airflow DAGs; PySpark → Scala Spark/Snowpark).

System design: clarify requirements → estimate scale → 5 stages (ingest/store/process/serve/orchestrate) → **cross-cutting** (idempotency, schema evolution, quality, backfills, observability, cost, security, recovery) → trade-offs.

Behavioral: STAR, ~90 seconds, **lead with the metric**, 50% on *your* Actions, always have a follow-up number ready.

A.5 Glossary of key terms

Term	One-line definition
ACID	Atomicity, Consistency, Isolation, Durability — transactional guarantees Delta adds to the lake.
AQE	Adaptive Query Execution — Spark 3+ runtime re-optimization (coalesce, broadcast, skew split).
CDC	Change Data Capture — propagate only inserts/updates/deletes from a source.
CDF	Change Data Feed — Delta's row-level change stream for incremental consumption.
Conformed dimension	A dimension shared consistently across facts so metrics drill across.
DAG	Directed Acyclic Graph — the dependency graph of orchestrated tasks.
Grain	The exact meaning of one fact-table row; declare it before modeling anything.
Idempotency	Re-running a job produces the same result with no duplicates or corruption.
Medallion	Bronze/Silver/Gold layering ≈ raw/clean/presentation.
MERGE	Upsert by key — the basis of idempotent, CDC-safe loads.
Micro-partition	Snowflake's automatic ~50–500 MB storage unit with min/max pruning metadata.
OLTP / OLAP	Transactional row-store source systems vs analytical columnar warehouses.
Sargable	A predicate that can use an index (`col = x`, not `FUNC(col) = x`).
SCD	Slowly Changing Dimension — strategies (Type 0–6) for handling attribute history.
Shuffle	Redistributing data across partitions by key — the main Spark cost.
Skew	Uneven key distribution overloading one task; fix with salting/AQE/broadcast.
Watermark	"I won't wait for data later than X" — bounds streaming state and late data.
Z-ORDER	Multi-dimensional clustering that improves Delta data skipping.

A.6 Suggested next deliverables (in order)

1. **AWS + GCP gap chapters** (Modules 6B/6C) — the single biggest coverage win.
2. **Master Question Bank + answer key** appendix for timed self-testing.
3. **Security & compliance module** (Module 11) — high-frequency, currently thin.
4. **Paged.js upgrade** for printed TOC page numbers, running headers, and an index.
5. **Printable flashcards + mock score tracker** to operationalize the 4-week plan.

You already have the experience and the core modules. Closing these gaps converts "well-prepared on my stack" into "prepared for any DE loop on the market."